PROBEY SERVICES · TRAINING GUIDE

Building Client Websites as WordPress Plugins

From an approved design to a website the client can manage alone, step by step, from zero.

Beginner to advanced · Folder structure · Dynamic content · Elementor · Plugin File Editor
Worked example: **My Cafe Site** · Real project: **Raining Berries**

October 2026

Contents

1. How to use this guide
2. WordPress, themes, plugins and Elementor
3. Tools you need
4. Plan before you code
5. The folder structure
6. Step 1: the main plugin file (your first plugin)
7. Step 2: constants and helper functions
8. Step 3: load the CSS and JavaScript
9. Step 4: header and footer
10. Step 5: page sections as HTML files
11. Step 6: create the pages automatically
12. Step 7: pictures the client can replace
13. Step 8: a dynamic list in the dashboard
14. Step 9: show the list on a page (templates + shortcode)
15. Step 10: a simple Settings screen
16. Step 11: starting content and the activation function
17. Step 12: JavaScript, SEO and mobile
18. Step 13: zip, install, update
19. Who changes what: the full map
20. Cookbook: common changes step by step
21. Make a static section dynamic (full lesson)
22. Advanced: more fields and options
23. Testing checklist
24. Fixing problems
25. Handing over to the client
26. Using AI tools (Antigravity) to build faster

A. The complete main file
B. Raining Berries: dynamic FAQ code
C. Glossary
D. One-page checklist

PART A - BASICS

CHAPTER 1

How to use this guide

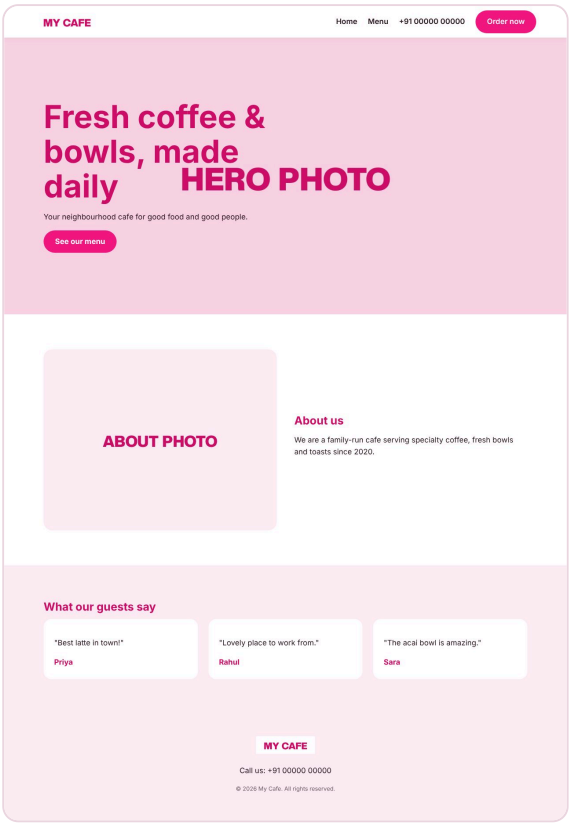
This guide teaches you, step by step, how to turn a client's approved design into a WordPress plugin that builds the whole website by itself, and that the client can manage without calling you.

You do not need to know anything about plugins yet. Every new word is explained the first time it appears, and there is a glossary at the end (Appendix C).

What you will build

We build one small but complete plugin together, called **My Cafe Site**. When you upload and activate it, it:

- creates the **Home** and **Menu** pages in Elementor, one Elementor section per design section;
- adds the **header and footer** to those pages;
- copies every **picture** into the Media Library and adds a **Pictures** screen with a Replace button;
- adds a **Menu Items** list in the dashboard, so the client can add, edit and delete dishes with photo, description and price;
- adds a **Settings** screen (phone number, "Order now" link);
- fills in **starting content**, so the site never looks empty.



The sample site built in this guide (placeholder pictures).

The real example: Raining Berries

Everything here is the same method we used for the **Raining Berries** website plugin. Where it helps, the guide shows the matching Raining Berries file, so you can open it in your own dashboard and compare.

How the guide is organised

Part	Chapters	What you learn
A. Basics	1 - 4	What a plugin is, the tools, and how to plan the work
B. Build from scratch	5 - 18	Every file and every step, in order, with the full code
C. Changing things	19 - 22	How to change anything later, and how to make a static section dynamic
D. Finish	23 - 26	Testing, fixing problems, handing over to the client, using AI tools
Appendix	A - D	Full code, the Raining Berries FAQ code, glossary, checklists

TIP

Every code example in this guide was tested

All code comes from a plugin that was installed and tested on a real WordPress site before this guide was written. Copy it exactly, including every bracket and semicolon. The sample plugin is also included as a zip next to this PDF, so you can compare your files with it.

CHAPTER 2

WordPress, themes, plugins and Elementor

A simple picture

Think of a website like a **restaurant**:

Website part	Restaurant	What it does
WordPress	The building	Stores everything (pages, pictures, users, settings) in a database and shows it to visitors.
Theme	The interior decoration	Decides how pages look. One theme is active at a time.
Plugin	An extra machine in the kitchen	Adds features. Many plugins can be active together.
Elementor	A plugin to arrange tables	Lets you build pages visually from sections and widgets.
Our plugin	The chef who sets up everything	Creates the pages, the header/footer, the dashboard lists and settings for one client.

Why a plugin and not a theme?

- The design and features stay safe when the client changes or updates the theme.
- One zip file installs the whole website in one click.
- Everything for one client lives in one folder, so it is easy to back up, copy and update.

The 4 languages inside a plugin

A plugin is just a folder of text files. They are written in four languages, and each has one job:

Language	File ends with	Job	Example in our plugin
PHP	.php	The brain. Runs on the server, reads the database, decides what to show.	Reads Menu Items and puts them in the page.
HTML	.html	The skeleton. The text, pictures and buttons in order.	The hero heading and button.
CSS	.css	The clothes. Colours, fonts, spacing, mobile layout.	Pink buttons, 3 cards per row.
JavaScript	.js	The movement. What happens when someone clicks or scrolls.	The mobile menu button opens the menu.

REMEMBER

The one sentence to remember

PHP brings the data from the dashboard, HTML is the design, CSS is the look, JavaScript is the clicks.

Static and dynamic content

- **Static** = typed once into an HTML file or Elementor section. Example: the "About us" paragraph.
- **Dynamic** = stored in the dashboard as a list and put into the page by PHP every time someone visits. Example: the menu items. The client adds a new dish in the dashboard and it appears on the website. Nobody touches the design.

CHAPTER 3

Tools you need

Tool	What for	Where
A WordPress site for testing (a "staging" site)	Never build on the client's live site. Make a test copy first.	Hostinger: Websites > Create or a subdomain like `test.yoursite.com`
Elementor (free)	Our pages are Elementor pages.	Plugins > Add New > search "Elementor"
A code editor	Writing the files on your computer.	VS Code (free) or Antigravity
Hostinger File Manager	Adding new files on the server, fixing problems.	hPanel > Websites > File Manager
Plugin File Editor	Small edits to existing plugin files from the dashboard.	WordPress: Tools > Plugin File Editor
A zip tool	WordPress installs plugins from a .zip file.	Mac: right-click folder > Compress. Windows: right-click > Send to > Compressed folder
Browser developer tools	Seeing errors, testing mobile size.	Chrome: right-click > Inspect. Phone icon = mobile view

5 golden rules

1. **Work on a staging site.** Move to the live site only when everything works.
2. **Back up before every change.** Download the plugin folder (File Manager > right-click > Compress > Download) or copy the code into a text file.
3. **Use a prefix.** Every function and setting name starts with the plugin's short name (`mcs_` for My Cafe Site, `rb_` for Raining Berries). Two plugins with the same function name crash the site.
4. **Raise the version number** every time you change the plugin (1.0.0 → 1.0.1). Browsers then download the new CSS/JS instead of an old copy.
5. **Change one thing at a time** and check the website after each change.

CAREFUL

What happens if PHP has a mistake?

A missing semicolon in a .php file can show "There has been a critical error on this website". Do not panic: Chapter 24 shows how to fix it in 1 minute with File Manager. WordPress also checks your file when you save in the Plugin File Editor and refuses to save if it finds a fatal error.

CHAPTER 4

Plan before you code

Most rework comes from not knowing what the client wants to change later. Spend 30 minutes planning with the approved design open.

Step 1: list every page and every section

Write each page and give each section a short name, top to bottom. These names become your file names.

Page	Sections (top to bottom)
Home	01-hero, 02-about, 03-reviews
Menu	01-title, then the menu items list

Step 2: mark each part static or dynamic

Ask the client one question for every section: **"Will you ever want to add one more of these?"** If yes, it is a **dynamic list**.

Part of the design	Changes how often?	Decision	How the client edits it
Hero heading and photo	Rarely	Static	Elementor + Pictures screen
About text	Rarely	Static	Elementor
Menu items	Often ("add a new drink")	**Dynamic list**	Dashboard > Menu Items
Reviews	Sometimes	Static now, dynamic in Chapter 21	Dashboard > Reviews
Phone number, order link	Sometimes, used in many places	**Setting**	My Cafe > Settings

For Raining Berries the same exercise gave: **dynamic** = Menu Items and categories; **settings** = order link, App Store links, popup type; **pictures** = Pictures screen; everything else static in Elementor.

Step 3: list the fields for each dynamic list

For every dynamic list, write the fields the client fills in. Keep it short: every extra field is something the client has to understand.

List	Fields
Menu Items	Name, Photo, Description, Price, Category, Order number
Reviews	Guest name, Review text, Order number

TIP

Questions to ask the client

- Which parts will you update yourself?
- Which lists will grow (products, team, branches, offers, FAQs)?
- Which values appear in many places (phone, email, address, order link, social links)?
- Who will update the site: you, or a staff member? (Keeps the dashboard as simple as possible.)

PART B - BUILD FROM SCRATCH

CHAPTER 5

The folder structure

Here is the complete plugin folder. **Create these folders now** on your computer, empty for the moment. We fill them in one by one in the next chapters.

```
my-cafe-site/                  ← the plugin folder (name = plugin name, lowercase, hyphens)
├── my-cafe-site.php           ← MAIN FILE: the "brain" (PHP). Same name as the folder.
├── README.txt                 ← install notes for you and the client
├── parts/                     ← pieces shown on EVERY page
│   ├── header.html
│   └── footer.html
├── pages/                     ← one folder per page, one file per section
│   ├── home/
│   │   ├── 01-hero.html
│   │   ├── 02-about.html
│   │   └── 03-reviews.html
│   └── menu/
│       └── 01-title.html
├── templates/                 ← designs for ONE item of a dynamic list
│   ├── menu-card.html
│   └── menu-group.html
├── assets/                    ← files the browser downloads
│   ├── css/site.css           ← design (colours, fonts, mobile rules)
│   ├── js/site.js             ← clicks and movement (menu button, sliders)
│   ├── img/                   ← pictures (imported into Media on activation)
│   └── fonts/                 ← custom fonts (if the design has them)
└── data/                      ← settings written as text
    ├── pages.json             ← which pages to create + their sections + SEO
    └── menu-seed.json         ← starting content for the dynamic list
```

Why this structure?

Folder	Rule	Why
`my-cafe-site.php`	Only PHP. No design here.	The brain stays readable and safe to edit.
`parts/`	Things on every page.	Change the header once, it changes everywhere.
`pages/<page>/`	One file per section, numbered 01, 02, 03 in page order.	Each file becomes one Elementor section, so the client can move or edit sections one by one.
`templates/`	The design of ONE item of a list, with blanks like `%NAME%`.	PHP copies it once per item and fills the blanks.
`assets/`	Everything the browser downloads.	Standard WordPress practice.
`data/`	Lists written as JSON text.	Add a page or starting content without touching PHP.

Naming rules

- Lowercase letters, numbers and hyphens only: `01-hero.html`, not `Hero Section.html`.
- The main PHP file has the **same name as the folder**: `my-cafe-site/my-cafe-site.php`.
- Picture names start with the prefix and say what they show: `mcs-hero.jpg`, `rb-logo.webp`.
- All PHP functions start with the prefix: `mcs_fill()`, `mcs_menu_html()`.

The same structure in Raining Berries

```
raining-berries-site/
├── raining-berries-site.php     (the brain)
├── parts/header.html, footer.html
├── pages/home/01-hero-banner.html ... 08-faq.html
├── pages/about/, menu/, locations/, nutrition/, locations-brandon/ ...
├── templates/menu-card.html, menu-group.html, menu-tab-on.html ...
├── assets/css/raining-berries.css, js/raining-berries.js, img/, fonts/
└── data/pages.json, menu-seed.json
```

CHAPTER 6

Step 1: the main plugin file (your first plugin)

Every plugin starts with one PHP file that has a special comment at the top called the **plugin header**. WordPress reads it to show the plugin in the Plugins list.

Create `my-cafe-site.php` and type this:

my-cafe-site.php - the beginning

```
<?php
/**
 * Plugin Name: My Cafe Site
 * Description: Sample client-site plugin. Creates the pages in Elementor, adds header + footer,
imports pictures and adds a "Menu Items" list the client can manage from the dashboard.
 * Version:     1.0.0
 * Author:      Probey Services
 * Requires PHP: 7.2
 */

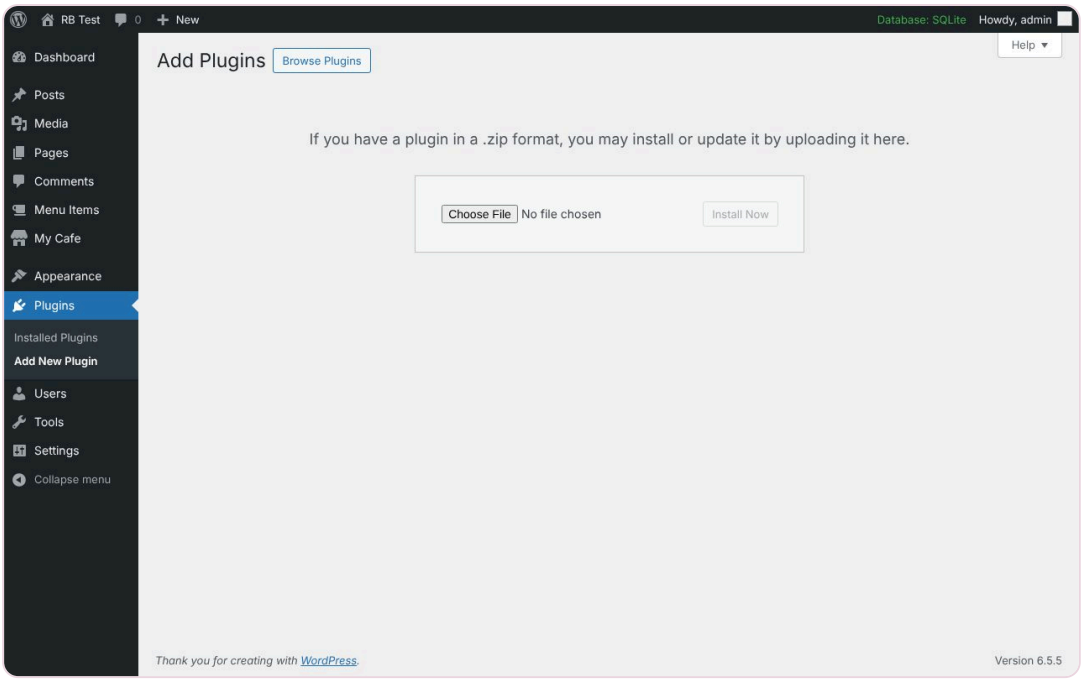
/* Stop if someone opens this file directly in the browser (security). */
if ( ! defined( 'ABSPATH' ) ) { exit; }
```

Line by line

Line	Meaning
`<?php`	Starts PHP code. Every .php file begins with it. Nothing (not even a space) before it.
`Plugin Name`	The name in Plugins > Installed Plugins. The only required line.
`Description, Version, Author`	Shown under the name. Raise Version for every update.
`Requires PHP`	Lowest PHP version the plugin needs.
`if ( ! defined( 'ABSPATH' ) ) { exit; }`	Security: if someone opens the file directly in a browser, stop. `ABSPATH` only exists when WordPress is running.

Try your first plugin now

1. Save the file inside the `my-cafe-site` folder.
2. Zip the **folder** (not just the file): you get `my-cafe-site.zip`.
3. WordPress dashboard: **Plugins > Add New > Upload Plugin** > Choose File > Install Now.
4. Click **Activate**. It does nothing yet, but it is your first working plugin.



Plugins > Add New > Upload Plugin

CAREFUL

"Destination folder already exists"

This means a folder with the same name is already on the server (an older copy or a broken upload). Plugins > Installed Plugins > Deactivate > Delete it first, or delete the folder in File Manager under `wp-content/plugins/`, then upload again. If the old copy is a working plugin, WordPress instead offers **"Replace current with uploaded"**, which is the normal way to update.

CHAPTER 7

Step 2: constants and helper functions

Next we add a few values and small functions that the rest of the plugin uses again and again.

Constants: fixed values

my-cafe-site.php - PART 1

```
/* ================================================================== */
/* PART 1. CONSTANTS - fixed values used everywhere                   */
/* ================================================================== */
define( 'MCS_VER', '1.0.0' );                          // raise this when you change the plugin
define( 'MCS_DIR', plugin_dir_path( __FILE__ ) );     // folder path on the server (for reading files)
define( 'MCS_URL', plugin_dir_url( __FILE__ ) );      // web address of the folder (for CSS, JS,
pictures)
```

- `MCS_DIR` is the folder on the server. Use it to **read** files: `MCS_DIR . 'parts/header.html'`.
- `MCS_URL` is the web address of the folder. Use it for files the **browser** loads (CSS, JS, pictures).
- The dot `.` joins two pieces of text together in PHP.

Helpers: small reusable functions

my-cafe-site.php - PART 2

```
/* ================================================================== */
/* PART 2. HELPERS - small functions we reuse                         */
/* ================================================================== */

/* Read a setting saved on the "My Cafe > Settings" page. */
function mcs_opt( $key, $default = '' ) {
  $all = get_option( 'mcs_settings', array() );
  return ( isset( $all[ $key ] ) && '' !== $all[ $key ] ) ? $all[ $key ] : $default;
}

/* Web address of a picture: the Media Library copy if it exists, otherwise the plugin copy. */
function mcs_img( $name ) {
  $map = get_option( 'mcs_images', array() );
  return isset( $map[ $name ]['url'] ) ? $map[ $name ]['url'] : MCS_URL . 'assets/img/' . $name;
}

/* Replace the placeholders in our HTML files with real values. */
function mcs_fill( $html ) {
  $html = str_replace( '{{HOME}}', trailingslashit( home_url( '/' ) ), $html );
  $html = str_replace( '{{YEAR}}', gmdate( 'Y' ), $html );
  $html = str_replace( '{{PHONE}}', esc_html( mcs_opt( 'phone', '+91 00000 00000' ) ), $html );
  $html = str_replace( '{{ORDER_URL}}', esc_url( mcs_opt( 'order_url', '#' ) ), $html );
  return preg_replace_callback( '/\{\{IMG:([^}}]+)\}\}/', function ( $m ) { return esc_url( mcs_img(
$m[1] ) ); }, $html );
}

/* Is the visitor on one of our pages? (made by this plugin) */
function mcs_is_our_page() {
  return ! is_admin() && is_singular( 'page' ) && get_post_meta( get_queried_object_id(), '_mcs_page',
true );
}
```

The placeholder idea (very important)

We never type the website address inside our HTML files, because it changes (staging → live). Instead we write a **placeholder** in double curly brackets, and `mcs_fill()` replaces it with the real value:

Placeholder in HTML	Becomes	Comes from
`{{HOME}}`	`https://yoursite.com/`	WordPress site address
`{{IMG:mcs-hero.jpg}}`	address of that picture in Media	Pictures screen (Chapter 12)
`{{PHONE}}`	+91 98765 43210	My Cafe > Settings (Chapter 15)
`{{ORDER_URL}}`	the order link	My Cafe > Settings
`{{YEAR}}`	2026	today's date

WHY?

Why escape values (esc_html, esc_url)?

Values typed by people can contain characters that break the page or are dangerous. `esc_html()` makes text safe to show, `esc_url()` makes a link safe, `esc_attr()` makes a value safe inside an attribute like `alt="..."`. Rule: **escape everything you print that came from the database or a form**.

CHAPTER 8

Step 3: load the CSS and JavaScript

WordPress has its own way to add CSS and JS files to a page, called **enqueue** (add to the queue). We load our files **only on our own pages**, so we never disturb the rest of the site.

my-cafe-site.php - load our files

```
add_action( 'wp_enqueue_scripts', function () {
  if ( ! mcs_is_our_page() ) { return; }
  wp_enqueue_style( 'mcs-site', MCS_URL . 'assets/css/site.css', array(), MCS_VER );
  wp_enqueue_script( 'mcs-site', MCS_URL . 'assets/js/site.js', array(), MCS_VER, true );
} );
```

What is add_action?

WordPress runs through fixed moments while it builds a page, called **hooks**. `add_action( 'moment', function )` means "when WordPress reaches this moment, also run my code".

Hook	When it runs	We use it for
`wp_enqueue_scripts`	While preparing the page's CSS/JS list	Adding site.css and site.js
`wp_body_open`	Right after <body> opens	Printing the header
`wp_footer`	Just before </body>	Printing the footer
`init`	Early, on every request	Creating dashboard lists
`admin_menu`	Building the dashboard sidebar	Adding "My Cafe" screens

A **filter** (`add_filter`) is similar, but it receives a value, may change it, and must give it back. Example: `body_class` receives the list of CSS classes of <body>, and we add `mcs-page`.

Switch off the theme's styles on our pages

The active theme has its own CSS for headings, links and buttons. It would change our design (for example, headings in a different font). So on our pages we remove the theme's stylesheets:

my-cafe-site.php - theme styles off + body class

```
/* The theme's own styles would change our design (fonts, colours, buttons): switch them off on our
pages. */
add_action( 'wp_enqueue_scripts', function () {
  if ( ! mcs_is_our_page() ) { return; }
  global $wp_styles;
  foreach ( (array) $wp_styles->queue as $handle ) {
    $src = (string) $wp_styles->registered[ $handle ]->src;
    if ( 0 === strpos( $src, get_template_directory_uri() ) || 0 === strpos( $src,
get_stylesheet_directory_uri() ) ) {
      wp_dequeue_style( $handle );
    }
  }
  wp_dequeue_style( 'global-styles' );      // block-theme colours and fonts
  wp_dequeue_style( 'wp-block-library' );   // block editor styles (not used on our pages)
```

```
}, 999 );

add_filter( 'body_class', function ( $classes ) {
  if ( mcs_is_our_page() ) { $classes[] = 'mcs-page'; }
  return $classes;
} );
```

TIP

Raining Berries does the same

In `raining-berries-site.php` look for *"The theme's own styles would fight the design"*. Without this, the client's theme could change fonts and button colours.

The CSS file

`assets/css/site.css` holds the whole look. Note the **variables** at the top: change `--pink` once and every pink button changes. The `@media (max-width: 800px)` block at the end is the **mobile layout**.

assets/css/site.css

```
/* My Cafe - design styles. Colours are set once here, as variables. */
:root { --pink: #F21781; --deep: #CE0E68; --blush: #FDECF4; --ink: #2A1620; }
body.mcs-page { margin: 0; font-family: system-ui, sans-serif; color: var(--ink); font-size: 17px;
line-height: 1.6; }

/* Elementor clean-up: no extra padding around our sections */
.mcs-sec .elementor-container, .mcs-sec .elementor-column, .mcs-sec .elementor-widget-wrap,
.mcs-sec .elementor-widget-container { padding: 0 !important; margin: 0 !important; max-width: none
!important; display: block; }
.mcs-sec .elementor-widget:not(:last-child) { margin-bottom: 0 !important; }

.mcs-wrap { max-width: 1140px; margin: 0 auto; padding: 0 20px; }
.mcs-btn { display: inline-block; background: var(--pink); color: #fff !important; padding: 12px 26px;
border-radius: 999px; font-weight: 700; text-decoration: none; }
.mcs-btn:hover { background: var(--deep); }
h1, h2, h3 { color: var(--deep); line-height: 1.1; margin: 0 0 14px; }

/* Header */
.mcs-header { position: sticky; top: 0; z-index: 50; background: #fff; box-shadow: 0 2px 12px
rgba(0,0,0,.06); }
.mcs-header__row { display: flex; align-items: center; justify-content: space-between; height: 72px; }
.mcs-logo img { height: 44px; display: block; }
.mcs-nav { display: flex; gap: 24px; align-items: center; }
.mcs-nav a { color: var(--ink); text-decoration: none; font-weight: 600; }
.mcs-burger { display: none; font-size: 28px; background: none; border: 0; cursor: pointer; }

/* Sections */
.mcs-hero { background: center / cover no-repeat; padding: 140px 0; }
.mcs-hero h1 { font-size: clamp(36px, 6vw, 72px); max-width: 12ch; }
.mcs-about, .mcs-reviews, .mcs-title { padding: 80px 0; }
.mcs-title { background: var(--blush); text-align: center; }
.mcs-grid-2 { display: grid; grid-template-columns: 1fr 1fr; gap: 40px; align-items: center; }
.mcs-grid-2 img { width: 100%; border-radius: 20px; }
.mcs-grid-3 { display: grid; grid-template-columns: repeat(3, 1fr); gap: 24px; }
.mcs-reviews { background: var(--blush); }
.mcs-review { background: #fff; border-radius: 18px; padding: 24px; margin: 0; }
.mcs-review cite { color: var(--deep); font-weight: 700; font-style: normal; }

/* Menu cards (filled from the dashboard) */
```

```
.mcs-menu { padding: 60px 0; max-width: 1140px; margin: 0 auto; }
.mcs-group { padding: 0 20px 50px; }
.mcs-card { background: #fff; border-radius: 20px; overflow: hidden; box-shadow: 0 10px 30px
rgba(206,14,104,.08); }
.mcs-card img { width: 100%; height: 200px; object-fit: cover; display: block; background: var(--
blush); }
.mcs-card__body { padding: 16px 18px; }
.mcs-price { font-weight: 800; color: var(--deep); }

/* Footer */
.mcs-footer { background: var(--blush); text-align: center; padding: 50px 0; }
.mcs-footer__logo { height: 40px; }
.mcs-small { font-size: 13px; opacity: .7; }

/* Mobile (phones and small tablets) */
@media (max-width: 800px) {
  .mcs-burger { display: block; }
  .mcs-nav { display: none; position: absolute; top: 72px; left: 0; right: 0; background: #fff; flex-
direction: column; padding: 20px; }
  .mcs-nav.is-open { display: flex; }
  .mcs-grid-2, .mcs-grid-3 { grid-template-columns: 1fr; }
  .mcs-hero { padding: 90px 0; text-align: center; }
}
```

CHAPTER 9

Step 4: header and footer

The header and footer are plain HTML files in `parts/`. PHP prints them on our pages and fills in the placeholders.

parts/header.html

```
<!-- HEADER (shown on every page). Placeholders like {{HOME}} are filled in by the plugin. -->
<header class="mcs-header">
  <div class="mcs-wrap mcs-header__row">
    <a href="{{HOME}}" class="mcs-logo"><img src="{{IMG:mcs-logo.png}}" alt="My Cafe"></a>
    <nav class="mcs-nav" id="mcs-nav">
      <a href="{{HOME}}">Home</a>
      <a href="{{HOME}}menu/">Menu</a>
      <a href="tel:{{PHONE}}">{{PHONE}}</a>
      <a href="{{ORDER_URL}}" class="mcs-btn">Order now</a>
    </nav>
    <button class="mcs-burger" data-mcs="menu-toggle" aria-label="Open menu">&#9776;</button>
  </div>
</header>
```

parts/footer.html

```
<!-- FOOTER (shown on every page) -->
<footer class="mcs-footer">
  <div class="mcs-wrap">
    <img src="{{IMG:mcs-logo.png}}" alt="My Cafe" class="mcs-footer__logo">
    <p>Call us: {{PHONE}}</p>
    <p class="mcs-small">&copy; {{YEAR}} My Cafe. All rights reserved.</p>
  </div>
</footer>
```

And the PHP that prints them:

my-cafe-site.php - print header and footer

```
/* Header: printed right after <body> opens. */
add_action( 'wp_body_open', function () {
  if ( mcs_is_our_page() ) { echo mcs_fill( file_get_contents( MCS_DIR . 'parts/header.html' ) ); } //
phpcs:ignore
} );

/* Footer: printed at the end of the page. */
add_action( 'wp_footer', function () {
  if ( mcs_is_our_page() ) { echo mcs_fill( file_get_contents( MCS_DIR . 'parts/footer.html' ) ); } //
phpcs:ignore
} );
```

- `file_get_contents()` reads a whole file as text.
- `mcs_fill()` replaces `{{HOME}}`, `{{PHONE}}` and the others.
- `echo` prints the result into the page.

WHY?

Why are the header and footer not inside Elementor?

They are the same on every page. Keeping them in one file means one change updates all pages. Because PHP fills them in on every visit, settings such as the phone number update instantly.

CHAPTER 10

Step 5: page sections as HTML files

Each section of the design becomes one small HTML file in `pages/<page>/`. Start each file with a comment that tells the next person what the section is and how to edit it.

pages/home/01-hero.html

```
<!-- SECTION 1: HERO. Edit the heading and the button text below. Picture: My Cafe > Pictures > mcs-hero.jpg -->
<section class="mcs-hero" style="background-image:url('{{IMG:mcs-hero.jpg}}')">
  <div class="mcs-wrap">
    <h1>Fresh coffee &amp;amp; bowls, made daily</h1>
    <p>Your neighbourhood cafe for good food and good people.</p>
    <a href="{{HOME}}menu/" class="mcs-btn">See our menu</a>
  </div>
</section>
```

pages/home/02-about.html

```
<!-- SECTION 2: ABOUT -->
<section class="mcs-about">
  <div class="mcs-wrap mcs-grid-2">
    <img src="{{IMG:mcs-about.jpg}}" alt="Inside My Cafe">
    <div>
      <h2>About us</h2>
      <p>We are a family-run cafe serving specialty coffee, fresh bowls and toasts since 2020.</p>
    </div>
  </div>
</section>
```

pages/home/03-reviews.html (static for now)

```
<!-- SECTION 3: REVIEWS (static: the text is typed here). Chapter 12 makes this dynamic. -->
<section class="mcs-reviews">
  <div class="mcs-wrap">
    <h2>What our guests say</h2>
    <div class="mcs-grid-3">
      <blockquote class="mcs-review"><p>"Best latte in town!"</p><cite>Priya</cite></blockquote>
      <blockquote class="mcs-review"><p>"Lovely place to work from."</p><cite>Rahul</cite>
</blockquote>
      <blockquote class="mcs-review"><p>"The acai bowl is amazing."</p><cite>Sara</cite></blockquote>
    </div>
  </div>
</section>
```

pages/menu/01-title.html

```
<!-- SECTION 1: PAGE TITLE -->
<section class="mcs-title">
  <div class="mcs-wrap">
    <h1>Our menu</h1>
    <p>Everything below comes from Dashboard &gt; Menu Items.</p>
  </div>
</section>
```

How to cut an approved design into sections

1. Open the approved HTML design in your code editor.
2. Find each top-level `<section>` (or each big block) from top to bottom.
3. Copy each block into its own file: `01-...`, `02-...` in the same order.
4. Replace the site address in links with `{{HOME}}` (e.g. `{{HOME}}menu/`).
5. Replace each picture address with `{{IMG:file-name.jpg}}` and put the picture in `assets/img/`.
6. Move all <style> CSS into `assets/css/site.css` and all <script> code into `assets/js/site.js`.

TIP

Big designs exported from a design tool

Some design tools export everything in one huge file, with pictures packed inside the code or the page drawn by JavaScript. For Raining Berries we opened the design in a browser, saved the finished page HTML, then cut it into sections. Ask an AI tool (Chapter 26) to do this cutting for you, then check every section.

CHAPTER 11

Step 6: create the pages automatically

Now we tell the plugin which pages to create. Instead of writing this in PHP, we write it in a simple text file, `data/pages.json`:

data/pages.json

```
{
  "home": {
    "title_short": "Home",
    "seo_title": "My Cafe | Specialty Coffee & Fresh Bowls",
    "seo_desc": "Family-run cafe serving specialty coffee, fresh bowls and toasts.",
    "sections": ["01-hero.html", "02-about.html", "03-reviews.html"]
  },
  "menu": {
    "title_short": "Menu",
    "seo_title": "Menu | My Cafe",
    "seo_desc": "Coffee, bowls and toasts. See our full menu.",
    "sections": ["01-title.html", "[mcs_menu]"]
  }
}
```

Key	Meaning
`"home"`, `"menu"`	The page address (slug): `yoursite.com/menu/`. Also the folder name inside `pages/`.
`title_short`	Page name in Pages > All Pages.
`seo_title`, `seo_desc`	The Google title and description (Chapter 17).
`sections`	The section files in order. Text in square brackets like `[mcs_menu]` is a **shortcode** (Chapter 14).

CAREFUL

JSON is strict

Use double quotes only. Put a comma between items but **not after the last one**. One wrong comma breaks the file. Check it on *jsonlint.com* if unsure.

The PHP that builds the Elementor pages

Elementor saves every page as a list of sections → columns → widgets, stored in the database as text (JSON) under the name `_elementor_data`. Our plugin writes this list itself: one full-width section per file, holding one **HTML widget** (or a **Shortcode widget** for `[...]`).

my-cafe-site.php - build Elementor sections + create pages

```
/* One Elementor section per HTML file (or a Shortcode widget for "[...]"). */
function mcs_elementor_sections( $slug, $files ) {
  $out = array();
```

```
  foreach ( $files as $n => $file ) {
    $id = substr( md5( $slug . $n ), 0, 7 );
    if ( '[' === $file[0] ) {
      $widget = array( 'id' => $id . 'w', 'elType' => 'widget', 'widgetType' => 'shortcode',
'settings' => array( 'shortcode' => $file ), 'elements' => array() );
    } else {
      $html   = mcs_fill( file_get_contents( MCS_DIR . 'pages/' . $slug . '/' . $file ) );
      $widget = array( 'id' => $id . 'w', 'elType' => 'widget', 'widgetType' => 'html', 'settings' =>
array( 'html' => $html ), 'elements' => array() );
    }
    $out[] = array(
      'id' => $id, 'elType' => 'section', 'isInner' => false,
      'settings' => array( 'layout' => 'full_width', 'gap' => 'no', 'css_classes' => 'mcs-sec',
'_title' => $file ),
      'elements' => array( array(
        'id' => $id . 'c', 'elType' => 'column', 'isInner' => false,
        'settings' => array( '_column_size' => 100 ),
        'elements' => array( $widget ),
      ) ),
    );
  }
  return $out;
}

function mcs_create_pages( $rebuild ) {
  $pages = json_decode( file_get_contents( MCS_DIR . 'data/pages.json' ), true );
  foreach ( $pages as $slug => $p ) {
    $page = get_page_by_path( $slug );
    if ( $page && ! $rebuild ) { continue; }                                  // page exists: keep
client edits
    if ( $page && ! get_post_meta( $page->ID, '_mcs_page', true ) ) { continue; } // not ours: never
touch
    $id = $page ? $page->ID : wp_insert_post( array( 'post_type' => 'page', 'post_status' =>
'publish', 'post_title' => $p['title_short'], 'post_name' => $slug ) );
    update_post_meta( $id, '_wp_page_template', 'elementor_canvas' );   // blank page, no theme header
    update_post_meta( $id, '_elementor_edit_mode', 'builder' );
    update_post_meta( $id, '_elementor_data', wp_slash( wp_json_encode( mcs_elementor_sections( $slug,
$p['sections'] ) ) ) );
    update_post_meta( $id, '_mcs_page', $slug );
    update_post_meta( $id, '_mcs_title', $p['seo_title'] );
    update_post_meta( $id, '_mcs_desc', $p['seo_desc'] );
    if ( 'home' === $slug ) { update_option( 'show_on_front', 'page' ); update_option(
'page_on_front', $id ); }
  }
}
```

The safety rules inside mcs_create_pages()

- **Page exists and we are not rebuilding?** Skip it, so the client's Elementor edits are never overwritten.
- **Page exists but was not made by us** (no `_mcs_page` mark)? Never touch it.
- `elementor_canvas` = a blank page template without the theme's header and footer; ours are added instead.
- The page marked `home` becomes the site's front page.

CHAPTER 12

Step 7: pictures the client can replace

The client must be able to change a picture without editing code. Our plan:

1. On activation, copy every file from `assets/img/` into the **Media Library**.
2. Remember each picture's new address in a saved list (an **option** called `mcs_images`).
3. Everywhere we need a picture we use `{{IMG:name}}`, which `mcs_img()` turns into that address.
4. A **Pictures** screen shows all pictures with a **Replace** button. Replacing updates the list and every page.

my-cafe-site.php - PART 7

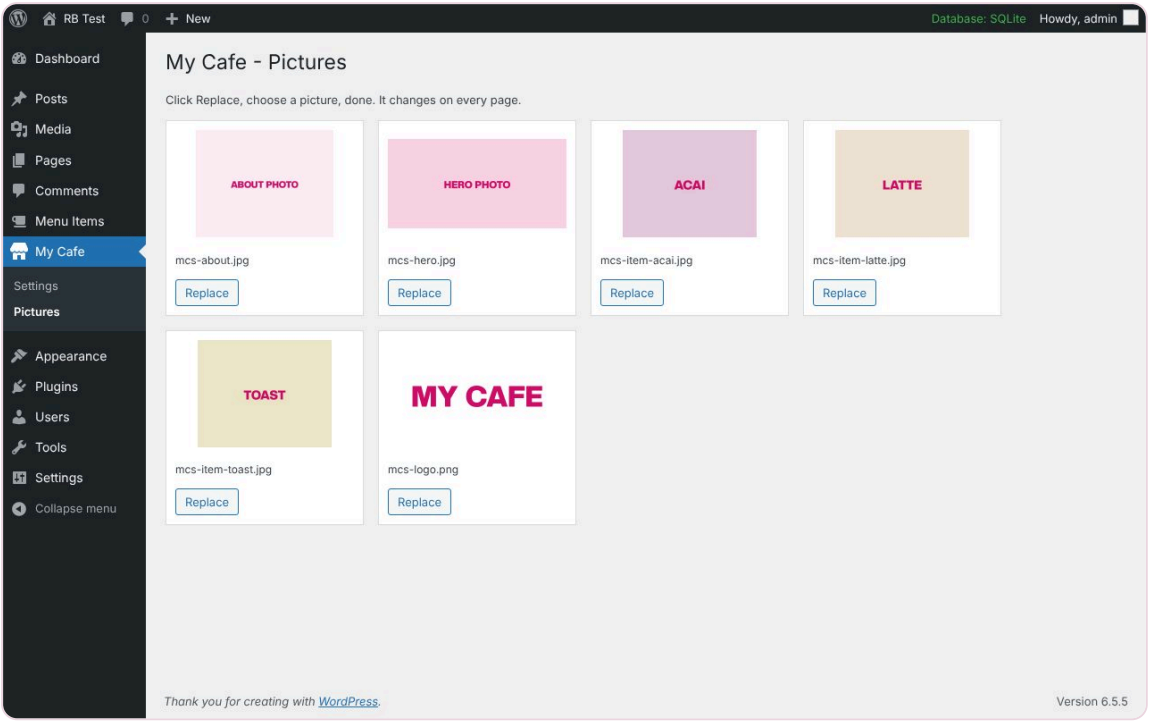
```
/* ================================================================== */
/* PART 7. PICTURES - import into Media + "Replace" page              */
/* ================================================================== */

function mcs_import_images() {
  require_once ABSPATH . 'wp-admin/includes/image.php';
  $map = get_option( 'mcs_images', array() );
  foreach ( glob( MCS_DIR . 'assets/img/*.*' ) as $path ) {
    $name = basename( $path );
    if ( isset( $map[ $name ]['id'] ) && get_post( $map[ $name ]['id'] ) ) { continue; }   // already
imported
    $up = wp_upload_bits( $name, null, file_get_contents( $path ) );
    if ( ! empty( $up['error'] ) ) { continue; }
    $type = wp_check_filetype( $name );
    $id   = wp_insert_attachment( array( 'post_mime_type' => $type['type'], 'post_title' => $name,
'post_status' => 'inherit' ), $up['file'] );
    wp_update_attachment_metadata( $id, wp_generate_attachment_metadata( $id, $up['file'] ) );
    $map[ $name ] = array( 'id' => $id, 'url' => $up['url'] );
  }
  update_option( 'mcs_images', $map, false );
}

function mcs_pictures_page() {
  if ( ! current_user_can( 'manage_options' ) ) { return; }
  $map = get_option( 'mcs_images', array() );
  if ( isset( $_POST['pic_name'], $_POST['pic_id'] ) && check_admin_referer( 'mcs_pics' ) ) {
    $name = sanitize_file_name( wp_unslash( $_POST['pic_name'] ) );
    $new  = wp_get_attachment_url( absint( $_POST['pic_id'] ) );
    if ( $new && isset( $map[ $name ] ) ) {
      $old = $map[ $name ]['url'];
      $map[ $name ] = array( 'id' => absint( $_POST['pic_id'] ), 'url' => $new );
      update_option( 'mcs_images', $map, false );
      mcs_replace_everywhere( $old, $new );
      echo '<div class="notice notice-success"><p>Picture replaced on every page.</p></div>';
    }
  }
  wp_enqueue_media();
  echo '<div class="wrap"><h1>My Cafe - Pictures</h1><p>Click Replace, choose a picture, done. It
changes on every page.</p>';
  echo '<form method="post" id="mcs-pic-form">';
  wp_nonce_field( 'mcs_pics' );
  echo '<input type="hidden" name="pic_name" id="pic_name"><input type="hidden" name="pic_id"
id="pic_id"></form>';
```

```
  echo '<div style="display:flex;flex-wrap:wrap;gap:16px">';
  foreach ( $map as $name => $m ) {
    echo '<div style="width:200px;background:#fff;padding:10px;border:1px solid #ddd">';
    echo '<img src="' . esc_url( $m['url'] ) . '" style="width:100%;height:120px;object-
fit:contain">';
    echo '<p style="font-size:12px">' . esc_html( $name ) . '</p>';
    echo '<button type="button" class="button mcs-replace" data-name="' . esc_attr( $name ) .
'">Replace</button></div>';
  }
  echo '</div></div>';
  echo "<script>jQuery(function($){ $('.mcs-replace').on('click',function(){ var
n=$(this).data('name'); var f=wp.media({title:'Replace '+n,multiple:false,library:{type:'image'}});
f.on('select',function(){ $('#pic_name').val(n);
$('#pic_id').val(f.state().get('selection').first().id); $('#mcs-pic-form').submit(); }); f.open();
}); });</script>";
}

/* Swap the old picture address for the new one inside every page we made. */
function mcs_replace_everywhere( $old, $new ) {
  $pages = get_posts( array( 'post_type' => 'page', 'posts_per_page' => -1, 'meta_key' => '_mcs_page',
'fields' => 'ids' ) ); // phpcs:ignore
  foreach ( $pages as $id ) {
    $data = json_decode( get_post_meta( $id, '_elementor_data', true ), true );
    if ( is_array( $data ) ) {
      array_walk_recursive( $data, function ( &$v ) use ( $old, $new ) { if ( is_string( $v ) ) { $v =
str_replace( $old, $new, $v ); } } );
      update_post_meta( $id, '_elementor_data', wp_slash( wp_json_encode( $data ) ) );
    }
    delete_post_meta( $id, '_elementor_css' );   // Elementor rebuilds its CSS
  }
}
```



My Cafe > Pictures: one Replace button per picture.

New function	What it does
`wp_upload_bits()`	Copies a file into the uploads folder.

New function	What it does
`wp_insert_attachment()`	Registers it in the Media Library.
`get_option()` / `update_option()`	Read / save a value in the database (our list of pictures).
`wp.media()`	Opens the normal WordPress media window to pick a picture.
`array_walk_recursive()`	Goes through every value of the Elementor data to swap the old address for the new one.

CLIENT CAN DO THIS

Replace a picture

My Cafe > Pictures > **Replace** > upload or choose a picture > **Select**. Done, on every page. Tip: use a picture with a similar shape (wide, square, tall) so the layout stays the same.

CHAPTER 13

Step 8: a dynamic list in the dashboard

This is the most important chapter. We create **Menu Items**, a list the client manages like posts. WordPress calls such a list a **Custom Post Type** (CPT). Its categories are a **Taxonomy**.

my-cafe-site.php - create the list + categories

```
add_action( 'init', 'mcs_register_lists' );
function mcs_register_lists() {
  /* The list itself (WordPress calls this a "custom post type"). */
  register_post_type( 'mcs_item', array(
    'labels'       => array(
      'name' => 'Menu Items', 'singular_name' => 'Menu Item', 'add_new_item' => 'Add Menu Item',
      'edit_item' => 'Edit Menu Item', 'featured_image' => 'Item photo', 'set_featured_image' => 'Set
item photo',
    ),
    'public'       => false,          // no separate page per item
    'show_ui'      => true,           // show it in the dashboard
    'menu_icon'    => 'dashicons-coffee',
    'supports'     => array( 'title', 'thumbnail', 'page-attributes' ), // name, photo, order number
  ) );
  /* Categories for the list (Coffee, Bowls...). */
  register_taxonomy( 'mcs_cat', 'mcs_item', array(
    'labels'            => array( 'name' => 'Categories', 'singular_name' => 'Category' ),
    'public'            => false,
    'show_ui'           => true,
    'show_admin_column' => true,
    'hierarchical'      => true,      // tick boxes, like post categories
  ) );
}
```

Setting	Meaning
`'mcs_item'`	The internal name. Max 20 characters, lowercase, use your prefix.
`labels`	The words the client sees. Use the client's language: "Menu Items", "Item photo".
`public => false`	Items do not get their own web page. They only appear where we put them.
`show_ui => true`	Show the list in the dashboard sidebar.
`menu_icon`	Sidebar icon. Search "WordPress dashicons" for the full list.
`supports`	`title` = name, `thumbnail` = photo, `page-attributes` = the Order number.
`hierarchical => true`	Categories appear as tick boxes.

Extra fields: a "meta box"

The list only has a name and photo by default. We add a box with **Description** and **Price**. Each value is saved as **post meta** (an extra value attached to the item).

my-cafe-site.php - fields + saving

```
/* Extra fields on the item screen: Description and Price. */
add_action( 'add_meta_boxes', function () {
  add_meta_box( 'mcs_item_box', 'Item details', 'mcs_item_box', 'mcs_item', 'normal', 'high' );
} );
function mcs_item_box( $post ) {
  wp_nonce_field( 'mcs_item_save', 'mcs_item_nonce' );   // security token for the form
  $desc  = get_post_meta( $post->ID, 'mcs_desc', true );
  $price = get_post_meta( $post->ID, 'mcs_price', true );
  echo '<p><strong>Description</strong><br><textarea name="mcs_desc" rows="3" style="width:100%">' .
esc_textarea( $desc ) . '</textarea></p>';
  echo '<p><strong>Price</strong> (for example 250)<br><input type="text" name="mcs_price" value="' .
esc_attr( $price ) . '" style="width:200px"></p>';
}

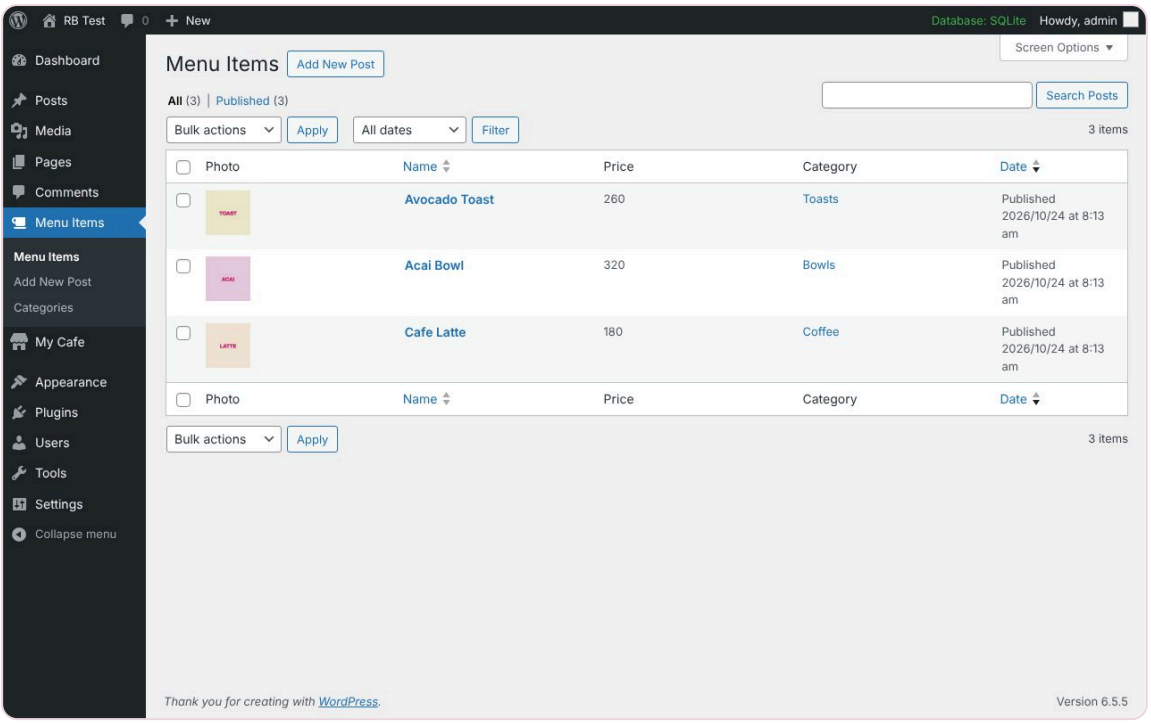
/* Save the fields when the client clicks Publish / Update. */
add_action( 'save_post_mcs_item', function ( $post_id ) {
  if ( ! isset( $_POST['mcs_item_nonce'] ) || ! wp_verify_nonce( sanitize_text_field( wp_unslash(
$_POST['mcs_item_nonce'] ) ), 'mcs_item_save' ) ) { return; }
  if ( ! current_user_can( 'edit_post', $post_id ) ) { return; }
  update_post_meta( $post_id, 'mcs_desc', sanitize_textarea_field( wp_unslash( isset(
$_POST['mcs_desc'] ) ? $_POST['mcs_desc'] : '' ) ) );
  update_post_meta( $post_id, 'mcs_price', sanitize_text_field( wp_unslash( isset( $_POST['mcs_price']
) ? $_POST['mcs_price'] : '' ) ) );
} );
```

Piece	Why it is there
`wp_nonce_field()` / `wp_verify_nonce()`	A secret token proving the form really came from this screen (security).
`current_user_can()`	Only people allowed to edit can save.
`sanitize_text_field()`	Cleans what was typed before saving it.
`update_post_meta()` / `get_post_meta()`	Save / read an extra value of an item.

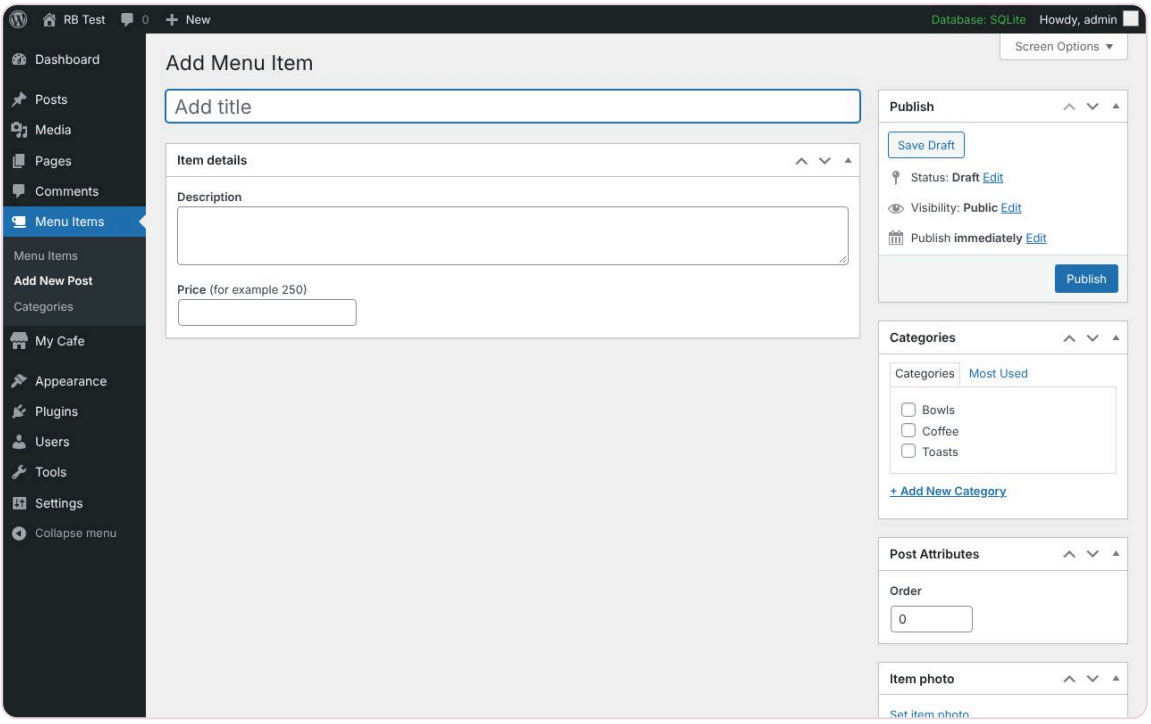
Make the list easy to scan

my-cafe-site.php - columns in the list

```
/* Photo + price columns in the list, so the client sees everything at a glance. */
add_filter( 'manage_mcs_item_posts_columns', function ( $cols ) {
  return array( 'cb' => $cols['cb'], 'mcs_photo' => 'Photo', 'title' => 'Name', 'mcs_price' =>
'Price', 'taxonomy-mcs_cat' => 'Category', 'date' => 'Date' );
} );
add_action( 'manage_mcs_item_posts_custom_column', function ( $col, $id ) {
  if ( 'mcs_photo' === $col ) { echo get_the_post_thumbnail( $id, array( 50, 50 ) ); }
  if ( 'mcs_price' === $col ) { echo esc_html( get_post_meta( $id, 'mcs_price', true ) ); }
}, 10, 2 );
```



Dashboard > Menu Items: photo, name, price and category at a glance.



Add Menu Item: name, Item details box, Categories, Order and photo.

CLIENT CAN DO THIS

Add a new menu item

Menu Items > Add Menu Item > type the name > fill Description and Price > tick a Category > Set item photo > **Publish**. It appears on the Menu page immediately.

CHAPTER 14

Step 9: show the list on a page (templates + shortcode)

Now PHP must turn the saved items into HTML with the exact design. We use two small **template** files with blanks:

templates/menu-group.html - one category

```
<div class="mcs-group" id="%SLUG%">
  <h2 class="mcs-group__title">%CAT%</h2>
  <div class="mcs-grid-3">%CARDS%</div>
</div>
```

templates/menu-card.html - one item

```
<article class="mcs-card">
  <img src="%IMG%" alt="%NAME%" loading="lazy">
  <div class="mcs-card__body">
    <h3>%NAME%</h3>
    <p>%DESC%</p>
    <span class="mcs-price">&#8377; %PRICE%</span>
  </div>
</article>
```

A **shortcode** is a short word in square brackets, like `[mcs_menu]`. When WordPress finds it on a page, it runs our function and puts the result there.

my-cafe-site.php - PART 5

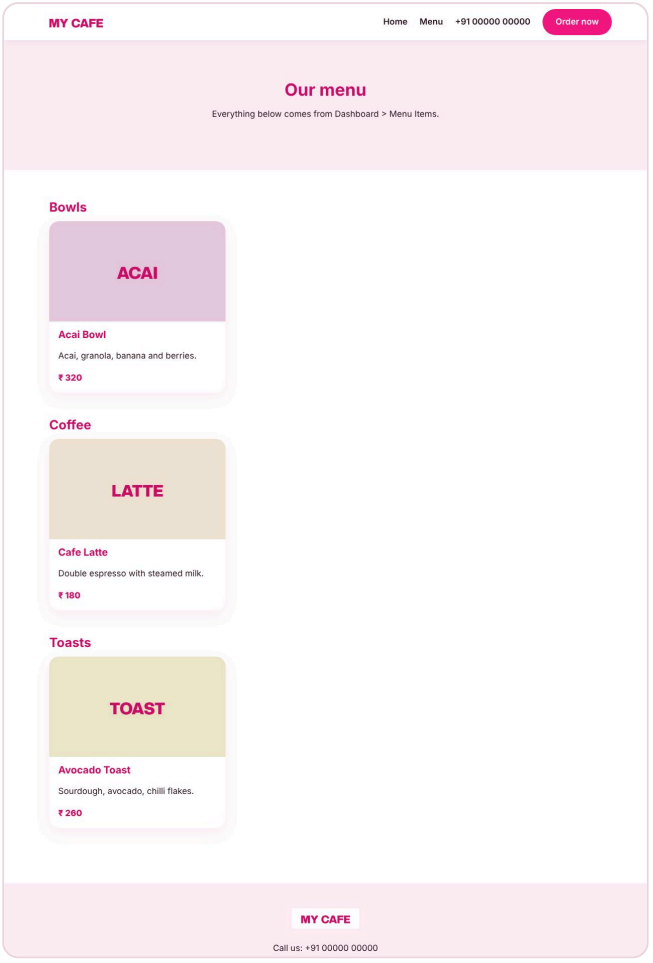
```
/* ================================================================== */
/* PART 5. SHORTCODE - shows the list on a page: [mcs_menu]           */
/* ================================================================== */

add_shortcode( 'mcs_menu', 'mcs_menu_html' );
function mcs_menu_html( $atts = array() ) {
  /* Options: [mcs_menu] shows everything, [mcs_menu category="coffee"] shows one category. */
  $atts      = shortcode_atts( array( 'category' => '' ), $atts );
  $card_tpl  = file_get_contents( MCS_DIR . 'templates/menu-card.html' );
  $group_tpl = file_get_contents( MCS_DIR . 'templates/menu-group.html' );
  $out = '';
  $args = array( 'taxonomy' => 'mcs_cat', 'hide_empty' => true, 'orderby' => 'name' );
  if ( $atts['category'] ) { $args['slug'] = sanitize_title( $atts['category'] ); }
  $cats = get_terms( $args );
  foreach ( $cats as $cat ) {
    $items = get_posts( array(
      'post_type'      => 'mcs_item',
      'posts_per_page' => -1,
      'orderby'        => array( 'menu_order' => 'ASC', 'title' => 'ASC' ),
      'tax_query'      => array( array( 'taxonomy' => 'mcs_cat', 'terms' => $cat->term_id ) ), //
phpcs:ignore
    ) );
    $cards = '';
    foreach ( $items as $item ) {
      $cards .= str_replace(
        array( '%IMG%', '%NAME%', '%DESC%', '%PRICE%' ),
```

```
        array(
          esc_url( get_the_post_thumbnail_url( $item->ID, 'medium_large' ) ),
          esc_html( get_the_title( $item ) ),
          esc_html( get_post_meta( $item->ID, 'mcs_desc', true ) ),
          esc_html( get_post_meta( $item->ID, 'mcs_price', true ) ),
        ),
        $card_tpl
      );
    }
    $out .= str_replace( array( '%CAT%', '%SLUG%', '%CARDS%' ), array( esc_html( $cat->name ),
esc_attr( $cat->slug ), $cards ), $group_tpl );
  }
  return '<div class="mcs-menu">' . $out . '</div>';
}
```

What happens when someone opens the Menu page

1. Elementor finds the Shortcode widget `[mcs_menu]` and runs `mcs_menu_html()`.
2. `get_terms()` gets the categories that have items.
3. For each category, `get_posts()` gets its items, sorted by the Order number, then name.
4. For each item, `str_replace()` copies `menu-card.html` and fills `%IMG%`, `%NAME%`, `%DESC%`, `%PRICE%`.
5. The cards go into `menu-group.html` (`%CARDS%`), and all groups are returned to the page.



The Menu page: every card comes from Dashboard > Menu Items.

REMEMBER

The pattern you will reuse for everything

List in dashboard → template with blanks → shortcode fills the blanks → Shortcode widget on the page. Team members, branches, offers, FAQs, testimonials: always the same 4 steps.

TIP

Raining Berries

Same pattern: `rb_menu_html()` fills `templates/menu-card.html`, `menu-group.html` and the tab buttons, and the Menu page has a Shortcode widget `[rb_menu]`.

CHAPTER 15

Step 10: a simple Settings screen

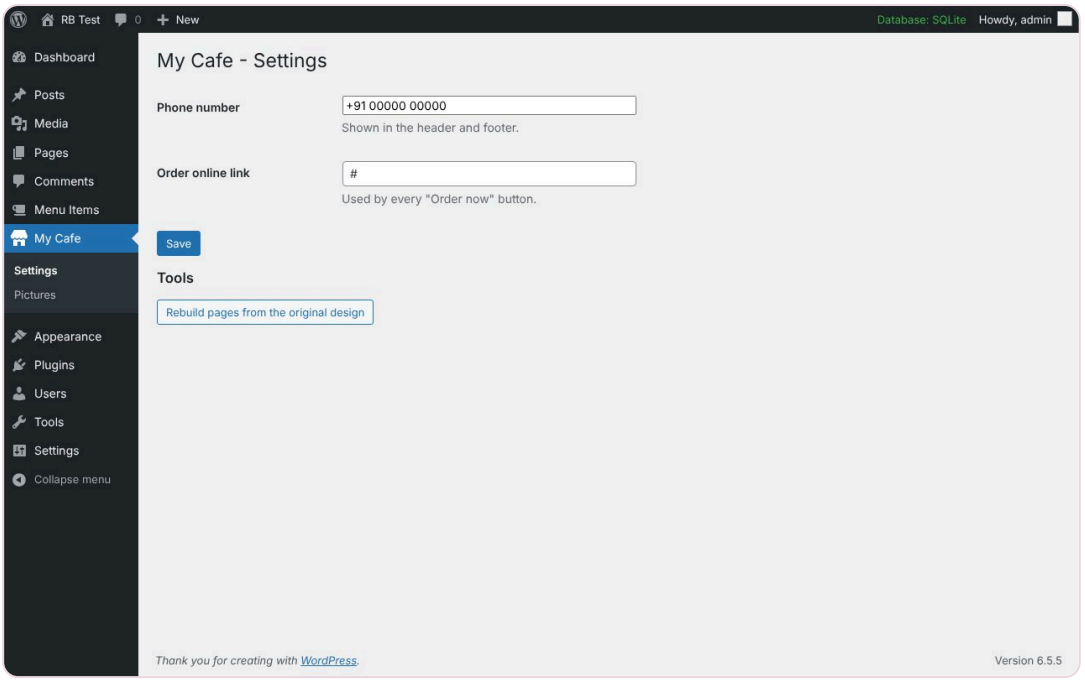
Values used in many places (phone, links) go in one Settings screen. The client changes it once and every page follows.

my-cafe-site.php - PART 6

```
/* ================================================================== */
/* PART 6. SETTINGS PAGE - My Cafe > Settings                         */
/* ================================================================== */

add_action( 'admin_menu', function () {
  add_menu_page( 'My Cafe', 'My Cafe', 'manage_options', 'mcs-settings', 'mcs_settings_page',
'dashicons-store', 58 );
  add_submenu_page( 'mcs-settings', 'Settings', 'Settings', 'manage_options', 'mcs-settings',
'mcs_settings_page' );
  add_submenu_page( 'mcs-settings', 'Pictures', 'Pictures', 'manage_options', 'mcs-pictures',
'mcs_pictures_page' );
} );

function mcs_settings_page() {
  if ( ! current_user_can( 'manage_options' ) ) { return; }
  if ( isset( $_POST['mcs_save'] ) && check_admin_referer( 'mcs_settings' ) ) {
    update_option( 'mcs_settings', array(
      'phone'     => sanitize_text_field( wp_unslash( $_POST['phone'] ) ),
      'order_url' => esc_url_raw( wp_unslash( $_POST['order_url'] ) ),
    ) );
    echo '<div class="notice notice-success"><p>Saved.</p></div>';
  }
  if ( isset( $_POST['mcs_rebuild'] ) && check_admin_referer( 'mcs_settings' ) ) {
    mcs_setup( true );
    echo '<div class="notice notice-success"><p>Pages rebuilt from the original design.</p></div>';
  }
  echo '<div class="wrap"><h1>My Cafe - Settings</h1><form method="post">';
  wp_nonce_field( 'mcs_settings' );
  echo '<table class="form-table">';
  echo '<tr><th>Phone number</th><td><input class="regular-text" name="phone" value="' . esc_attr(
mcs_opt( 'phone' ) ) . '"><p class="description">Shown in the header and footer.</p></td></tr>';
  echo '<tr><th>Order online link</th><td><input class="regular-text" type="url" name="order_url"
value="' . esc_attr( mcs_opt( 'order_url' ) ) . '"><p class="description">Used by every "Order now"
button.</p></td></tr>';
  echo '</table><p><button class="button button-primary" name="mcs_save" value="1">Save</button></p>';
  echo '<h2>Tools</h2><p><button class="button" name="mcs_rebuild" value="1" onclick="return
confirm(\'Rebuild all pages? Your Elementor edits on them will be lost.\')">Rebuild pages from the
original design</button></p>';
  echo '</form></div>';
}
```



My Cafe > Settings

Function	Does
`add_menu_page()`	Adds "My Cafe" to the sidebar.
`add_submenu_page()`	Adds "Settings" and "Pictures" under it.
`check_admin_referer()`	The security check for the form.
`update_option( 'mcs_settings', ... )`	Saves all settings together in one option.
`mcs_opt( 'phone' )`	Our helper to read one setting anywhere.

CAREFUL

Where can a setting placeholder be used?

Placeholders such as `{{PHONE}}` are filled **every time** in the header, footer and templates. Inside Elementor sections they are filled **once**, when the page is created. So: put values that change (phone, links) in the header, footer, templates or shortcodes, not inside Elementor text. Raining Berries fills App Store links inside Elementor pages on every visit with an extra filter (`elementor/frontend/the_content`), which you can copy if you need it.

CHAPTER 16

Step 11: starting content and the activation function

A new site with an empty menu looks broken. So on activation we add starting items from `data/menu-seed.json`, **only once**, so we never overwrite the client's own items.

data/menu-seed.json

```
[
  { "name": "Cafe Latte", "category": "Coffee", "desc": "Double espresso with steamed milk.", "price":
"180", "photo": "mcs-item-latte.jpg" },
  { "name": "Acai Bowl",  "category": "Bowls",  "desc": "Acai, granola, banana and berries.", "price":
"320", "photo": "mcs-item-acai.jpg" },
  { "name": "Avocado Toast", "category": "Toasts", "desc": "Sourdough, avocado, chilli flakes.",
"price": "260", "photo": "mcs-item-toast.jpg" }
]
```

my-cafe-site.php - activation + starting content

```
register_activation_hook( __FILE__, 'mcs_setup' );

function mcs_setup( $rebuild = false ) {
  if ( ! get_option( 'permalink_structure' ) ) {   // nice addresses like /menu/ instead of ?
page_id=14
    update_option( 'permalink_structure', '/%postname%/' );
  }
  mcs_register_lists();          // make sure the list exists before adding items
  mcs_import_images();           // 1. pictures into Media
  mcs_seed_items();              // 2. starting menu items (only the first time)
  mcs_create_pages( $rebuild );  // 3. pages in Elementor
  add_option( 'mcs_settings', array( 'phone' => '+91 00000 00000', 'order_url' => '#' ) );
  flush_rewrite_rules();
}

/* Starting content from data/menu-seed.json, so the site never looks empty. */
function mcs_seed_items() {
  if ( get_option( 'mcs_seeded' ) ) { return; }   // never twice: client changes stay safe
  $map  = get_option( 'mcs_images', array() );
  $seed = json_decode( file_get_contents( MCS_DIR . 'data/menu-seed.json' ), true );
  foreach ( $seed as $item ) {
    $term = term_exists( $item['category'], 'mcs_cat' );
    if ( ! $term ) { $term = wp_insert_term( $item['category'], 'mcs_cat' ); }
    $id = wp_insert_post( array( 'post_type' => 'mcs_item', 'post_status' => 'publish', 'post_title'
=> $item['name'] ) );
    wp_set_object_terms( $id, array( (int) $term['term_id'] ), 'mcs_cat' );
    update_post_meta( $id, 'mcs_desc', $item['desc'] );
    update_post_meta( $id, 'mcs_price', $item['price'] );
    if ( isset( $map[ $item['photo'] ]['id'] ) ) { set_post_thumbnail( $id, $map[ $item['photo'] ]
['id'] ); }
  }
  update_option( 'mcs_seeded', 1 );
}
```

What happens when you click Activate

1. `register_activation_hook` runs `mcs_setup()` once.
2. Nice page addresses are switched on (`/menu/` instead of `?page_id=14`).
3. The list type is created, pictures are imported, starting items are added (once), pages are created.
4. Default settings are saved with `add_option()`, which never overwrites an existing value.

TIP

The "Rebuild pages" button

Settings has a **Rebuild pages from the original design** button that calls `mcs_setup( true )`. Use it after you change the section files, or if the client breaks a page in Elementor. It overwrites Elementor edits on our pages, so it asks for confirmation first.

CHAPTER 17

Step 12: JavaScript, SEO and mobile

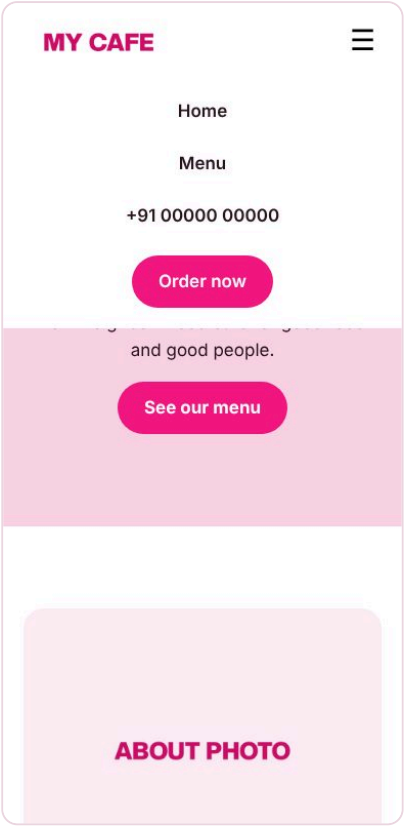
JavaScript: clicks and movement

JavaScript runs in the visitor's browser after the page has loaded. It never loads data; it only reacts to clicks and scrolling.

assets/js/site.js

```
/* My Cafe - clicks and small interactions (no data is loaded here). */
document.addEventListener('DOMContentLoaded', function () {
  /* Mobile menu: the burger button opens and closes the navigation. */
  var btn = document.querySelector('[data-mcs="menu-toggle"]');
  var nav = document.getElementById('mcs-nav');
  if (btn && nav) {
    btn.addEventListener('click', function () { nav.classList.toggle('is-open'); });
  }
});
```

Note the **hook attribute** `data-mcs="menu-toggle"` in the header HTML. JavaScript finds elements by these attributes. **Never delete** `data-...` **attributes** when editing HTML, or the button stops working. (Raining Berries uses `data-rb-click`, `data-rbif` and others the same way.)



Mobile: the menu button opens the navigation.

SEO basics built in

my-cafe-site.php - SEO title and description

```
/* SEO title + description saved on each page. */
add_filter( 'pre_get_document_title', function ( $title ) {
  $t = mcs_is_our_page() ? get_post_meta( get_queried_object_id(), '_mcs_title', true ) : '';
  return $t ? $t : $title;
} );
add_action( 'wp_head', function () {
  $d = mcs_is_our_page() ? get_post_meta( get_queried_object_id(), '_mcs_desc', true ) : '';
  if ( $d ) { echo '<meta name="description" content="' . esc_attr( $d ) . '">' . "\n"; }
} );
```

- Each page gets its **title** and **meta description** from `pages.json`.
- One **H1** per page (the main heading), H2 for sections, H3 for cards.
- Every picture has an **alt** text describing it.
- If the client uses Yoast or Rank Math, Raining Berries also writes the same title and description into their fields during setup.

Mobile

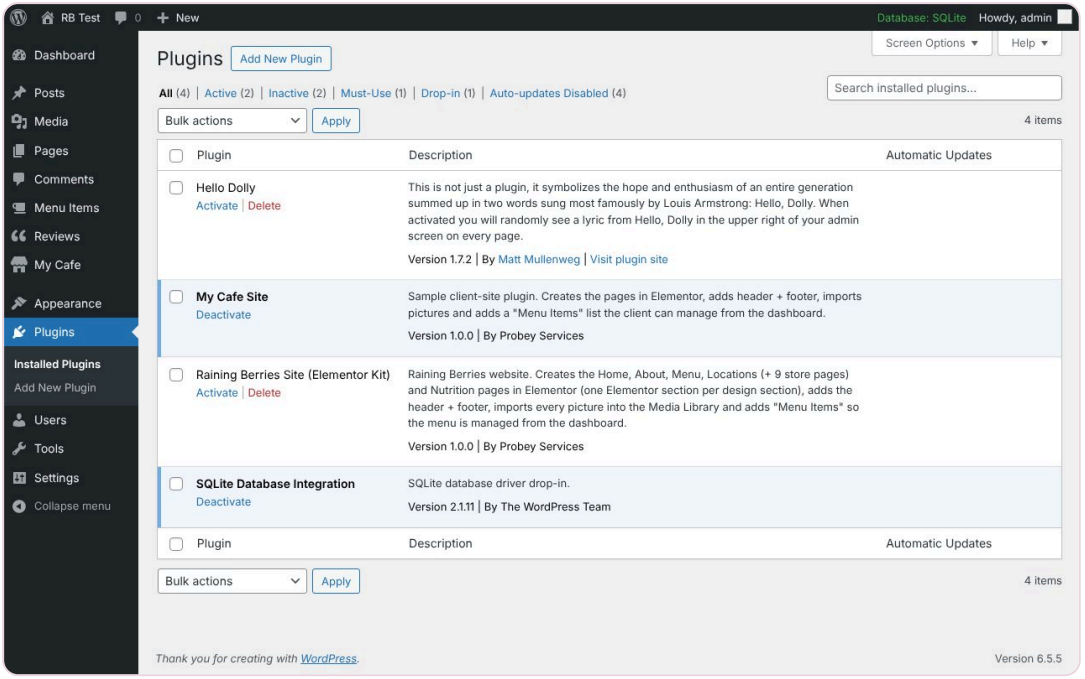
All mobile rules live in the `@media (max-width: 800px)` block at the end of `site.css`. Always test at phone width (Chrome > Inspect > phone icon) after every change.

CHAPTER 18

Step 13: zip, install, update

First install

1. Check every file is in place (compare with the folder tree in Chapter 5).
2. Zip the **folder** `my-cafe-site`.
3. Install and activate **Elementor** first.
4. Plugins > Add New > Upload Plugin > your zip > Install Now > **Activate** (takes 10-40 seconds while pictures are copied).
5. Open the site: Home and Menu should look like the design.



The plugin in Plugins > Installed Plugins.

Updating the plugin later

1. Make your changes on your computer and **raise the Version** (top of the main file and `MCS_VER`).
2. Zip the folder again with the **same folder name**.
3. Upload Plugin > choose the zip > WordPress says the plugin is already installed > **Replace current with uploaded**.
4. Clear caches: LiteSpeed Cache > Purge All, and Elementor > Tools > Regenerate CSS.

CAREFUL

Big zip files

If the upload stops halfway, the host's upload limit is too small. Upload the zip to `wp-content/plugins/` with File Manager, right-click > Extract, delete the zip, then activate in Plugins.

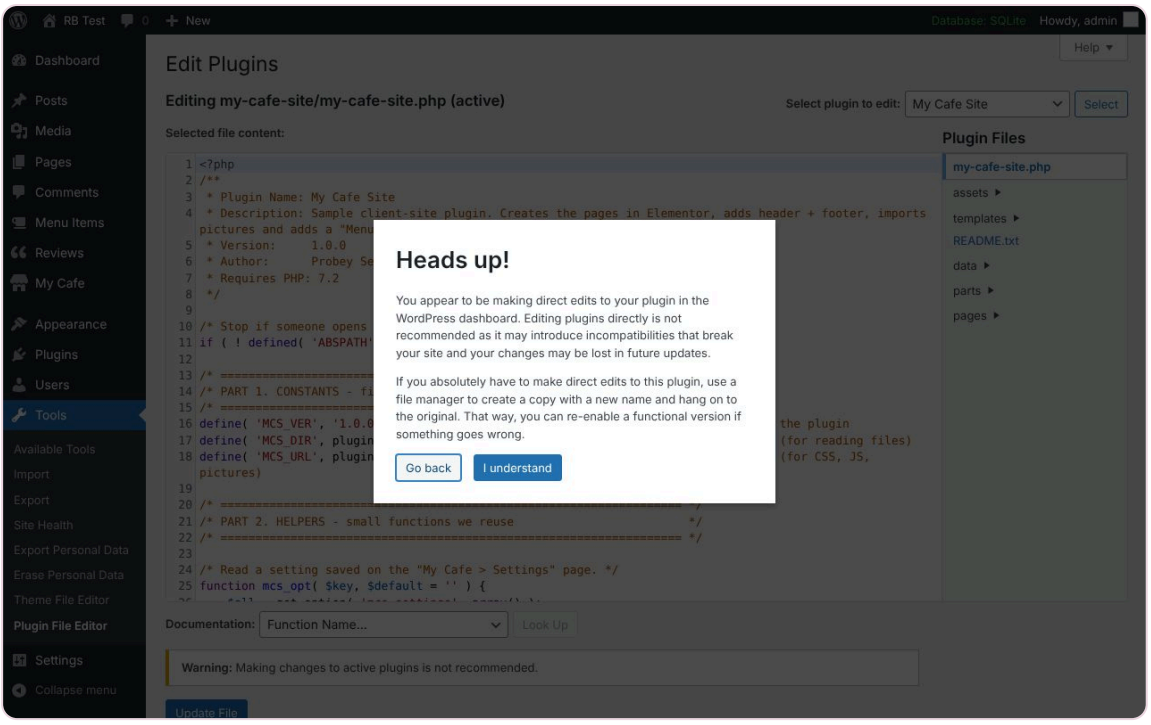
PART C - CHANGING THINGS

CHAPTER 19

Who changes what: the full map

Print this table for every project. It answers "how do I change X?" for you and for the client.

I want to change…	Who	Where	Code?
A menu item (add, edit, delete, photo, price)	Client	Dashboard > Menu Items	No
The order of items	Client	Edit item > Page Attributes > Order	No
Categories	Client	Menu Items > Categories	No
Phone number, order link	Client	My Cafe > Settings	No
Any picture on the pages	Client	My Cafe > Pictures > Replace	No
Text in a section (heading, paragraph)	Client	Pages > Edit with Elementor > HTML widget	Tiny (edit text between tags)
Move or remove a section on one page	Client	Elementor: drag or delete the section	No
A link in the header or footer	You	Plugin File Editor > `parts/header.html` / `footer.html`	HTML
The design of a menu card	You	Plugin File Editor > `templates/menu-card.html`	HTML
Colours, fonts, spacing, mobile layout	You	Plugin File Editor > `assets/css/site.css`	CSS
A new field for menu items (e.g. "Vegan")	You	Plugin File Editor > main .php file	PHP (Chapter 22)
A static section becomes a dashboard list	You	File Manager + Plugin File Editor	PHP (Chapter 21)
A brand new page	You	New folder in `pages/` + `pages.json` + Rebuild	HTML + JSON



Tools > Plugin File Editor: choose the plugin at the top right, then a file from the list.

CAREFUL

Two limits of the Plugin File Editor

1. It can only **edit existing files**. It cannot create a new file. Use Hostinger **File Manager** to create files (Chapter 21 shows how).
2. Changes are live immediately. Copy the file's full text into a notepad **before** you change it, so you can paste it back.

CHAPTER 20

Cookbook: common changes step by step

1. Change text in a section (client)

1. Pages > Home > **Edit with Elementor**.
2. Click the section, then its **HTML** widget on the left.
3. Change only the words between `>` and `<`, e.g. `<h1>Fresh coffee</h1>`.
4. Click **Update**, then view the page.

2. Change a link in the header (developer)

1. Tools > Plugin File Editor > select **My Cafe Site** (top right).
2. Click `parts/header.html` in the file list.
3. Find the link, e.g. `<a href="{{HOME}}menu/">Menu</a>`. Change the text or the address. To add a link, copy a whole `<a>...</a>` line and edit the copy.
4. **Update File**. Check the site, then copy the change into your local files too.

3. Change the main colour (developer)

1. Plugin File Editor > `assets/css/site.css`.
2. At the top change `--pink: #F21781;` to the new colour.
3. Update File. Raise the version (so browsers load the new CSS): edit `MCS_VER` in the main file.
4. Purge the cache.

4. Change the design of the menu cards (developer)

1. Plugin File Editor > `templates/menu-card.html`.
2. Change the HTML, but **keep the blanks** `%IMG%`, `%NAME%`, `%DESC%`, `%PRICE%`.
3. Update File. Every card changes at once.

5. Show only one category somewhere (developer or client)

Our shortcode has an option. In Elementor add a **Shortcode** widget and type:

```
[mcs_menu category="coffee"]
```

`coffee` is the category **slug** (see Menu Items > Categories). `[mcs_menu]` without the option shows everything. This is how the option is read (already in PART 5):

inside mcs_menu_html()

```
/* Options: [mcs_menu] shows everything, [mcs_menu category="coffee"] shows one category. */
  $atts      = shortcode_atts( array( 'category' => '' ), $atts );
```

6. Add a new section to a page

1. In Elementor: copy (duplicate) a similar section, change its HTML. Fastest, only for that page.
2. **Or, permanently in the plugin:** create `pages/home/04-offers.html` with File Manager, add `"04-offers.html"` to the `sections` list in `data/pages.json`, then My Cafe > Settings > **Rebuild pages**.

7. Add a new page

1. File Manager: create folder `pages/contact/` and the file `01-title.html` inside it.
2. Add a `"contact": { ... }` block in `data/pages.json` (copy the "menu" block, change the values, mind the commas).
3. Settings > **Rebuild pages** (or deactivate + activate the plugin). The page appears at `/contact/`.
4. Add a link to it in `parts/header.html`.

CHAPTER 21

Make a static section dynamic (full lesson)

The **Reviews** section on our home page is static: the three reviews are typed inside `pages/home/03-reviews.html`. The client wants to add new reviews from the dashboard. We make it dynamic in **5 steps**, all inside the plugin.

Step	What	Tool
1	Create the review card template file	Hostinger File Manager
2	Paste the new code at the end of the main PHP file	Plugin File Editor
3	Tell pages.json to use the shortcode	Plugin File Editor
4	Swap the section on the live page	Elementor (or Rebuild)
5	Add the reviews in the dashboard	Client

Step 1: create the template file (File Manager)

Look at the static HTML. One review is one `<blockquote>`. That is our template; replace the changing parts with blanks:

templates/review-card.html (new file)

```
<blockquote class="mcs-review"><p>"%TEXT%"</p><cite>%NAME%</cite></blockquote>
```

1. hPanel > Websites > your site > **File Manager**.
2. Open `public_html/wp-content/plugins/my-cafe-site/templates/`.
3. Click **New File**, name it `review-card.html`, open it, paste the line above, **Save**.

Step 2: paste the code (Plugin File Editor)

Tools > Plugin File Editor > My Cafe Site > `my-cafe-site.php`. Scroll to the **very end** of the file and paste this whole block after the last line. Click **Update File**.

paste at the END of my-cafe-site.php

```
/* ================================================================== */
/* PART 9. REVIEWS - a second dynamic list (added in Chapter 12)      */
/* ================================================================== */

/* 1. The list in the dashboard */
add_action( 'init', function () {
  register_post_type( 'mcs_review', array(
    'labels'    => array( 'name' => 'Reviews', 'singular_name' => 'Review', 'add_new_item' => 'Add
Review', 'edit_item' => 'Edit Review' ),
    'public'    => false,
    'show_ui'   => true,
    'menu_icon' => 'dashicons-format-quote',
    'supports'  => array( 'title', 'page-attributes' ),   // title = guest name
```

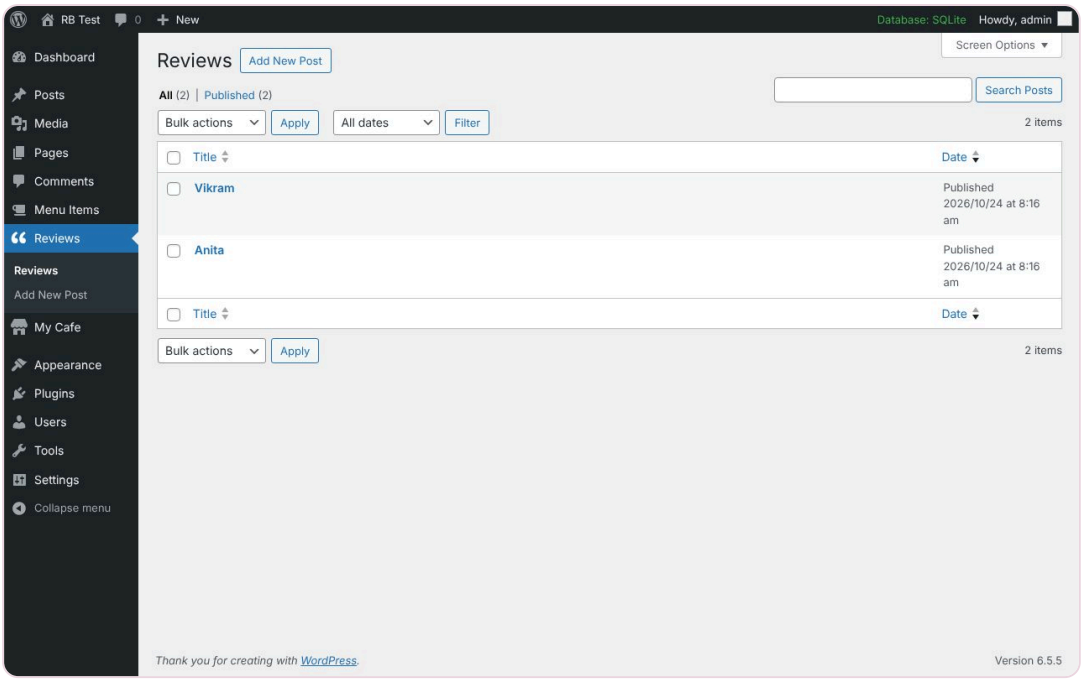
```
  ) );
} );

/* 2. One extra field: the review text */
add_action( 'add_meta_boxes', function () {
  add_meta_box( 'mcs_review_box', 'Review text', function ( $post ) {
    wp_nonce_field( 'mcs_review_save', 'mcs_review_nonce' );
    echo '<textarea name="mcs_review_text" rows="4" style="width:100%">' . esc_textarea(
get_post_meta( $post->ID, 'mcs_review_text', true ) ) . '</textarea>';
    echo '<p class="description">The title above is the guest\'s name.</p>';
  }, 'mcs_review', 'normal', 'high' );
} );

/* 3. Save the field */
add_action( 'save_post_mcs_review', function ( $post_id ) {
  if ( ! isset( $_POST['mcs_review_nonce'] ) || ! wp_verify_nonce( sanitize_text_field( wp_unslash(
$_POST['mcs_review_nonce'] ) ), 'mcs_review_save' ) ) { return; }
  if ( ! current_user_can( 'edit_post', $post_id ) ) { return; }
  update_post_meta( $post_id, 'mcs_review_text', sanitize_textarea_field( wp_unslash( isset(
$_POST['mcs_review_text'] ) ? $_POST['mcs_review_text'] : '' ) ) );
} );

/* 4. The shortcode [mcs_reviews] prints the whole section with the same design */
add_shortcode( 'mcs_reviews', function () {
  $card  = file_get_contents( MCS_DIR . 'templates/review-card.html' );
  $cards = '';
  $reviews = get_posts( array( 'post_type' => 'mcs_review', 'posts_per_page' => -1, 'orderby' =>
array( 'menu_order' => 'ASC', 'date' => 'DESC' ) ) );
  foreach ( $reviews as $r ) {
    $cards .= str_replace(
      array( '%TEXT%', '%NAME%' ),
      array( esc_html( get_post_meta( $r->ID, 'mcs_review_text', true ) ), esc_html( get_the_title( $r
) ) ),
      $card
    );
  }
  return '<section class="mcs-reviews"><div class="mcs-wrap"><h2>What our guests say</h2><div
class="mcs-grid-3">' . $cards . '</div></div></section>';
} );
```

What each part does is exactly what you learned in Chapters 13 and 14: **(1)** the list, **(2)** the field, **(3)** saving, **(4)** the shortcode that fills the template. The shortcode prints the whole section with the same classes as the static HTML, so it looks identical.



A new "Reviews" item appears in the dashboard sidebar.

Step 3: update pages.json (so Rebuild uses it)

Plugin File Editor > `data/pages.json`. In the home `sections` list replace `"03-reviews.html"` with `"[mcs_reviews]"`:

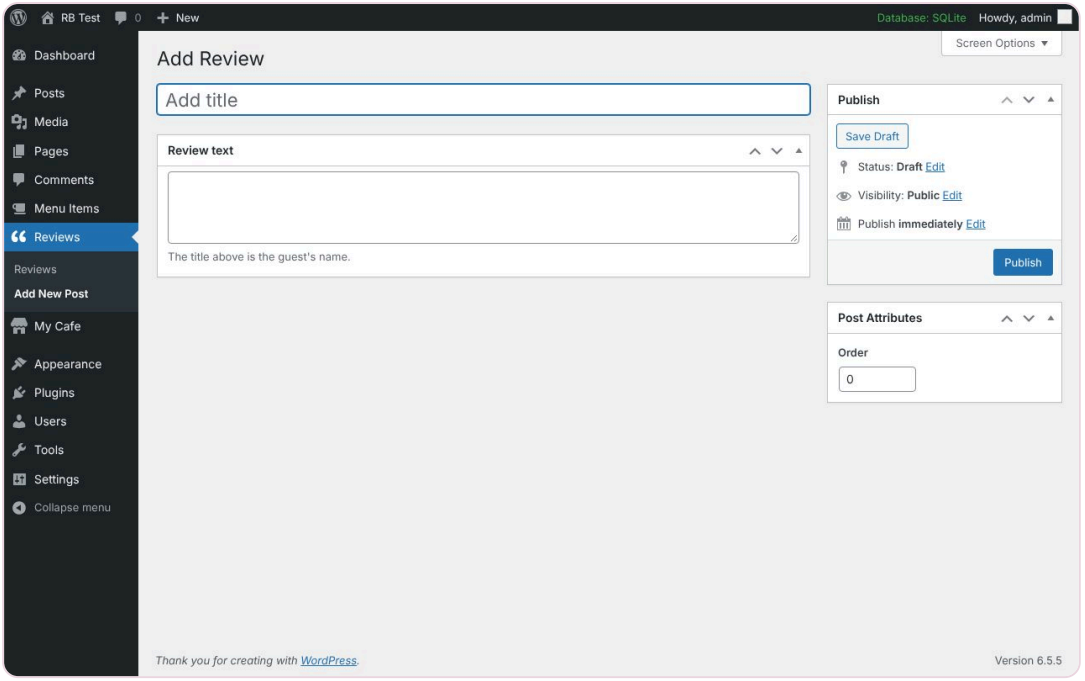
```
"sections": ["01-hero.html", "02-about.html", "[mcs_reviews]"]
```

Step 4: swap the section on the live page

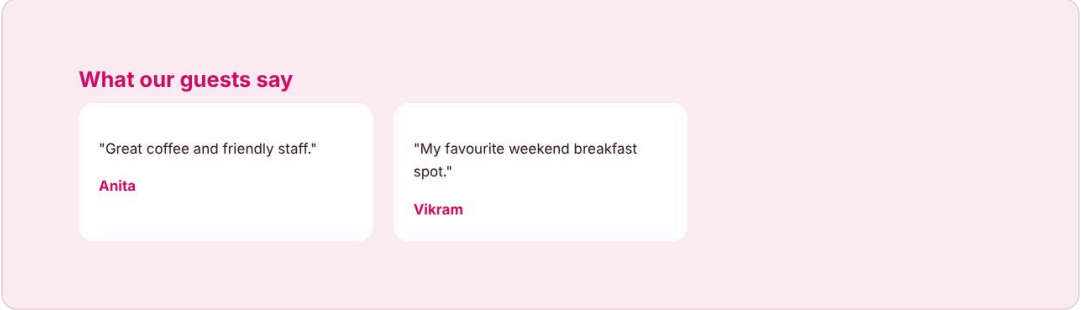
Choose **one** way:

- **A. Keep other Elementor edits:** Pages > Home > Edit with Elementor > delete the Reviews section > drag a **Shortcode** widget to the same place > type `[mcs_reviews]` > Update.
- **B. Fastest:** My Cafe > Settings > **Rebuild pages** (overwrites Elementor edits on our pages).

Step 5: the client adds reviews



Reviews > Add Review: the title is the guest name, the box is the review.



The same design, now filled from the dashboard.

TRY IT

Practise the pattern

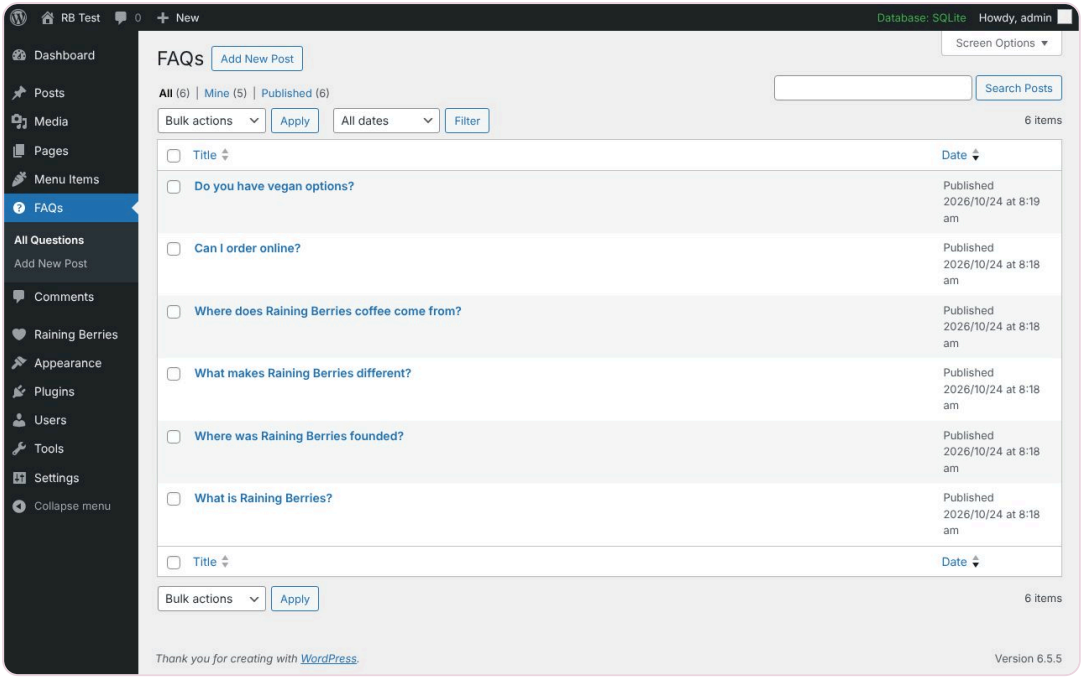
Do the same for an "Offers" section: list `mcs_offer`, fields Title + Text + Picture, template `offer-card.html`, shortcode `[mcs_offers]`. If you can do this alone, you can make any section of any client site dynamic.

The real case: the Raining Berries FAQ

The "Good to know" FAQ on the Raining Berries home page is static. Appendix B has the tested code that makes it a **Dashboard > FAQs** list with exactly the same design and the same open/close click. The steps are the same 5:

1. File Manager: add `templates/faq-item.html` and `templates/faq-wrap.html` (Appendix B) to `raining-berries-site/templates/`.
2. Plugin File Editor: paste the Appendix B code at the end of `raining-berries-site.php`.

3. `data/pages.json`: in the home sections replace `"08-faq.html"` with `"[rb_faq]"`.
4. Elementor: replace the FAQ section's HTML widget with a Shortcode widget `[rb_faq]` (or Raining Berries > Settings > Rebuild).
5. Open the dashboard once: the 5 existing questions are copied into FAQs automatically. Add more with **FAQs > Add Question**.



Raining Berries with the new FAQs list.

REMEMBER

Keep the data-... attributes

The FAQ templates keep `data-rb-click="toggleFaq"` and `data-rbif="f.open"` from the original HTML. That is why the existing JavaScript keeps opening and closing the answers without any change.

CHAPTER 22

Advanced: more fields and options

Add a new field to an existing list

Example: a "Vegan" tick box for menu items. Three small edits in `my-cafe-site.php`:

1. Show the field: inside `mcs_item_box()`, after the Price line, add:

```
  $vegan = get_post_meta( $post->ID, 'mcs_vegan', true );
  echo '<p><label><input type="checkbox" name="mcs_vegan" value="1"' . checked( $vegan, '1', false ) .
'> Vegan</label></p>';
```

2. Save it: inside the `save_post_mcs_item` function, after the price line, add:

```
  update_post_meta( $post_id, 'mcs_vegan', empty( $_POST['mcs_vegan'] ) ? '' : '1' );
```

3. Show it on the card: in `mcs_menu_html()` add a blank `%VEGAN%` to both lists of `str_replace`, with the value:

```
get_post_meta( $item->ID, 'mcs_vegan', true ) ? '<span class="mcs-vegan">Vegan</span>' : '',
```

...and put `%VEGAN%` where you want it in `templates/menu-card.html`. Add a style for `.mcs-vegan` in site.css.

Common field types

Field	HTML to show it	Clean it with
Short text	`<input type="text">`	`sanitize_text_field()`
Long text	`<textarea>`	`sanitize_textarea_field()`
Link	`<input type="url">`	`esc_url_raw()`
Number	`<input type="number">`	`absint()` or `floatval()`
Tick box	`<input type="checkbox" value="1">`	`empty() ? "" : "1"`
Email	`<input type="email">`	`sanitize_email()`
Picture	Use the built-in **photo** (thumbnail) of the list	`get_the_post_thumbnail_url()`

Shortcode options

Options make one shortcode useful in many places, like `[mcs_menu category="coffee"]` (Chapter 20). Read them with `shortcode_atts()` and use them in the query.

Order categories

Raining Berries adds a "Position on the menu page" number to each category (term meta `rb_order`) and sorts by it in `rb_menu_terms()`. Copy that code when the client needs categories in a fixed order

instead of A-Z.

Filters and tabs with JavaScript

Raining Berries prints all categories at once, and JavaScript hides the others when a tab is clicked (`setMenuCat()` in `raining-berries.js`). The data still comes from PHP; JS only shows and hides.

Header links from the dashboard

Raining Berries fills the header's Menu drop-down from the categories: the header file contains `{{MENU_DROP}}`, and PHP replaces it with one link per category (`rb_nav_links()`). A new category appears in the header automatically.

PART D - FINISH

CHAPTER 23

Testing checklist

Test on the staging site before every handover and every update.

Design

- [ ] Every page matches the approved design on desktop (1440 px wide).
- [ ] Tablet (768 px) and phone (390 px): nothing cut off, no sideways scrolling, text readable.
- [ ] Fonts load (not a default font) and pictures load (no broken icons).

Features

- [ ] Every button and link goes to the right place.
- [ ] Menus, sliders, tabs, popups and forms work, on phone too.
- [ ] The browser console (right-click > Inspect > Console) shows no red errors.

Dashboard (do it as the client would)

- [ ] Add, edit and delete an item in every list: the page updates.
- [ ] Change every setting: the site updates.
- [ ] Replace a picture: it changes on every page.
- [ ] Deactivate and activate the plugin: no duplicate pages or items.

Technical

- [ ] One H1 per page, alt text on pictures, page titles and descriptions.
- [ ] Works with the cache plugin on (LiteSpeed on Hostinger): purge, then re-check.
- [ ] Admin bar visible when logged in does not cover the header.

CHAPTER 24

Fixing problems

Problem	Why	Fix
"There has been a critical error" after editing PHP	A mistake in the .php file.	File Manager > `wp-content/plugins/` > rename the plugin folder (e.g. add `-off`). The site works again without the plugin. Fix the file (paste your backup), rename the folder back, activate.
"Destination folder already exists" on upload	An old or half-uploaded copy is there.	Delete it (Plugins list or File Manager), upload again.
Change does not show	Cache.	LiteSpeed Cache > Purge All; Elementor > Tools > Regenerate CSS; raise the plugin version; hard-refresh the browser (Ctrl/Cmd + Shift + R).
Pages show "404 Not found"	Page addresses not refreshed.	Settings > Permalinks > Post name > Save.
Menu page shows `[mcs_menu]` as text	The shortcode is inside an HTML widget, or the plugin is off.	Use a **Shortcode** widget; check the plugin is active.
Hover effects or mobile layout missing	A cache plugin removed "unused" CSS.	Exclude the plugin's CSS file from "Remove Unused CSS" in LiteSpeed.
Buttons/sliders do nothing	JS delayed by cache, or a `data-...` attribute was deleted.	Exclude the plugin's JS from "Delay JS"; restore the original HTML of that section.
Theme font or colours appear	Theme styles not switched off.	Check the theme-styles code (Chapter 8) is in the plugin.
Upload stops halfway	Upload size limit.	Upload with File Manager and Extract (Chapter 18).
A picture did not change after Replace	That page was made outside the plugin, or a cache.	Purge cache; for pages not made by the plugin, change the picture in Elementor.

TIP

Read the error yourself

Add `define( 'WP_DEBUG', true ); define( 'WP_DEBUG_LOG', true );` in `wp-config.php` (staging only). Errors are then written to `wp-content/debug.log` with the file name and line number.

CHAPTER 25

Handing over to the client

Give the client a one-page guide in their words. Copy this and adjust the names:

How to update your website

Log in: yoursite.com/wp-admin

- **Add or change a menu item:** Menu Items > Add Menu Item (name, description, price, category, photo) > Publish.
- **Change the order:** open the item > Order (1 = first) > Update.
- **Change a picture:** My Cafe > Pictures > Replace.
- **Change phone or order link:** My Cafe > Settings > Save.
- **Change text on a page:** Pages > the page > Edit with Elementor > click the text area > change only the words > Update.
- **Something looks wrong?** Do not delete anything. Call us; we can restore the original design in one click.

Handover checklist

- [ ] Client has an **Editor** account (not Administrator) if they should only edit content.
- [ ] Backup of the final plugin zip saved in the project folder.
- [ ] One-page guide sent.
- [ ] Pending items listed (e.g. "App Store links to add", "form emails to set up").

CHAPTER 26

Using AI tools (Antigravity) to build faster

AI coding tools can write most of the plugin from the approved design if you give clear instructions. You stay the teacher who checks the work.

Starting prompt

```
Build a WordPress plugin for Elementor from this approved HTML design.
- Plugin folder and main file: client-site/client-site.php, prefix "cs_".
- Keep the CSS exactly the same as the design. Put it in assets/css/site.css.
- Pages to create: Home, About, Menu. Split each page into one HTML file per
  section in pages/<page>/01-name.html. Each file = one Elementor section
  with an HTML widget. Pages use the Elementor Canvas template.
- Header and footer in parts/header.html and parts/footer.html, printed by PHP
  on our pages only. Switch off the theme's styles on our pages.
- Use placeholders {{HOME}} and {{IMG:file}} in all HTML files.
- Import all pictures in assets/img into the Media Library on activation and
  add a "Pictures" admin screen with a Replace button that updates every page.
- Dynamic lists (custom post types with simple meta boxes):
  * Menu Items: name, photo, description, price, category, order.
  Render with shortcode [cs_menu] using templates/menu-card.html with %BLANKS%.
- Settings screen: phone, order link. Seed starting content once.
- Never create duplicate pages or items when the plugin is re-activated.
```

Then work section by section

1. Install the result on staging and compare each section with the design.
2. Give feedback for one section at a time, with a screenshot.
3. Ask the AI to explain any code you do not understand, in simple words.
4. Run the testing checklist (Chapter 23) before handover.

CAREFUL

Check what the AI gives you

AI tools can make confident mistakes. Always test on staging. Every list must be tested by adding, editing and deleting an item. Every setting must be tested by changing it.

APPENDIX

APPENDIX A

The complete main file

The full, tested `my-cafe-site.php` (before the Reviews lesson). Compare your file with it line by line.

my-cafe-site/my-cafe-site.php

```
<?php
/**
 * Plugin Name: My Cafe Site
 * Description: Sample client-site plugin. Creates the pages in Elementor, adds header + footer,
imports pictures and adds a "Menu Items" list the client can manage from the dashboard.
 * Version:     1.0.0
 * Author:      Probey Services
 * Requires PHP: 7.2
 */

/* Stop if someone opens this file directly in the browser (security). */
if ( ! defined( 'ABSPATH' ) ) { exit; }

/* ================================================================== */
/* PART 1. CONSTANTS - fixed values used everywhere                   */
/* ================================================================== */
define( 'MCS_VER', '1.0.0' );                        // raise this when you change the plugin
define( 'MCS_DIR', plugin_dir_path( __FILE__ ) );     // folder path on the server (for reading files)
define( 'MCS_URL', plugin_dir_url( __FILE__ ) );      // web address of the folder (for CSS, JS,
pictures)

/* ================================================================== */
/* PART 2. HELPERS - small functions we reuse                         */
/* ================================================================== */

/* Read a setting saved on the "My Cafe > Settings" page. */
function mcs_opt( $key, $default = '' ) {
  $all = get_option( 'mcs_settings', array() );
  return ( isset( $all[ $key ] ) ) && '' !== $all[ $key ] ) ? $all[ $key ] : $default;
}

/* Web address of a picture: the Media Library copy if it exists, otherwise the plugin copy. */
function mcs_img( $name ) {
  $map = get_option( 'mcs_images', array() );
  return isset( $map[ $name ]['url'] ) ? $map[ $name ]['url'] : MCS_URL . 'assets/img/' . $name;
}

/* Replace the placeholders in our HTML files with real values. */
function mcs_fill( $html ) {
  $html = str_replace( '{{HOME}}', trailingslashit( home_url( '/' ) ), $html );
  $html = str_replace( '{{YEAR}}', gmdate( 'Y' ), $html );
  $html = str_replace( '{{PHONE}}', esc_html( mcs_opt( 'phone', '+91 00000 00000' ) ), $html );
  $html = str_replace( '{{ORDER_URL}}', esc_url( mcs_opt( 'order_url', '#' ) ), $html );
  return preg_replace_callback( '/\{\{IMG:([^}]+)\}\}/', function ( $m ) { return esc_url( mcs_img(
$m[1] ) ); }, $html );
}

/* Is the visitor on one of our pages? (made by this plugin) */
function mcs_is_our_page() {
  return ! is_admin() && is_singular( 'page' ) && get_post_meta( get_queried_object_id(), '_mcs_page',
true );
```

```
}

/* ================================================================== */
/* PART 3. FRONT END - CSS, JS, header and footer                     */
/* ================================================================== */

add_action( 'wp_enqueue_scripts', function () {
  if ( ! mcs_is_our_page() ) { return; }
  wp_enqueue_style( 'mcs-site', MCS_URL . 'assets/css/site.css', array(), MCS_VER );
  wp_enqueue_script( 'mcs-site', MCS_URL . 'assets/js/site.js', array(), MCS_VER, true );
} );

/* The theme's own styles would change our design (fonts, colours, buttons): switch them off on our
pages. */
add_action( 'wp_enqueue_scripts', function () {
  if ( ! mcs_is_our_page() ) { return; }
  global $wp_styles;
  foreach ( (array) $wp_styles->queue as $handle ) {
    $src = (string) $wp_styles->registered[ $handle ]->src;
    if ( 0 === strpos( $src, get_template_directory_uri() ) || 0 === strpos( $src,
get_stylesheet_directory_uri() ) ) {
      wp_dequeue_style( $handle );
    }
  }
  wp_dequeue_style( 'global-styles' );      // block-theme colours and fonts
  wp_dequeue_style( 'wp-block-library' );   // block editor styles (not used on our pages)
}, 999 );

add_filter( 'body_class', function ( $classes ) {
  if ( mcs_is_our_page() ) { $classes[] = 'mcs-page'; }
  return $classes;
} );

/* Header: printed right after <body> opens. */
add_action( 'wp_body_open', function () {
  if ( mcs_is_our_page() ) { echo mcs_fill( file_get_contents( MCS_DIR . 'parts/header.html' ) ); } //
phpcs:ignore
} );

/* Footer: printed at the end of the page. */
add_action( 'wp_footer', function () {
  if ( mcs_is_our_page() ) { echo mcs_fill( file_get_contents( MCS_DIR . 'parts/footer.html' ) ); } //
phpcs:ignore
} );

/* SEO title + description saved on each page. */
add_filter( 'pre_get_document_title', function ( $title ) {
  $t = mcs_is_our_page() ? get_post_meta( get_queried_object_id(), '_mcs_title', true ) : '';
  return $t ? $t : $title;
} );
add_action( 'wp_head', function () {
  $d = mcs_is_our_page() ? get_post_meta( get_queried_object_id(), '_mcs_desc', true ) : '';
  if ( $d ) { echo '<meta name="description" content="' . esc_attr( $d ) . '">' . "\n"; }
} );

/* ================================================================== */
/* PART 4. DYNAMIC LIST - "Menu Items" in the dashboard               */
/* ================================================================== */

add_action( 'init', 'mcs_register_lists' );
function mcs_register_lists() {
  /* The list itself (WordPress calls this a "custom post type"). */
  register_post_type( 'mcs_item', array(
    'labels'       => array(
```

```
      'name' => 'Menu Items', 'singular_name' => 'Menu Item', 'add_new_item' => 'Add Menu Item',
      'edit_item' => 'Edit Menu Item', 'featured_image' => 'Item photo', 'set_featured_image' => 'Set
item photo',
    ),
    'public'       => false,          // no separate page per item
    'show_ui'      => true,           // show it in the dashboard
    'menu_icon'    => 'dashicons-coffee',
    'supports'     => array( 'title', 'thumbnail', 'page-attributes' ), // name, photo, order number
  ) );
  /* Categories for the list (Coffee, Bowls...). */
  register_taxonomy( 'mcs_cat', 'mcs_item', array(
    'labels'            => array( 'name' => 'Categories', 'singular_name' => 'Category' ),
    'public'            => false,
    'show_ui'           => true,
    'show_admin_column' => true,
    'hierarchical'      => true,      // tick boxes, like post categories
  ) );
}

/* Extra fields on the item screen: Description and Price. */
add_action( 'add_meta_boxes', function () {
  add_meta_box( 'mcs_item_box', 'Item details', 'mcs_item_box', 'mcs_item', 'normal', 'high' );
} );
function mcs_item_box( $post ) {
  wp_nonce_field( 'mcs_item_save', 'mcs_item_nonce' );   // security token for the form
  $desc  = get_post_meta( $post->ID, 'mcs_desc', true );
  $price = get_post_meta( $post->ID, 'mcs_price', true );
  echo '<p><strong>Description</strong><br><textarea name="mcs_desc" rows="3" style="width:100%">' .
esc_textarea( $desc ) . '</textarea></p>';
  echo '<p><strong>Price</strong> (for example 250)<br><input type="text" name="mcs_price" value="' .
esc_attr( $price ) . '" style="width:200px"></p>';
}

/* Save the fields when the client clicks Publish / Update. */
add_action( 'save_post_mcs_item', function ( $post_id ) {
  if ( ! isset( $_POST['mcs_item_nonce'] ) || ! wp_verify_nonce( sanitize_text_field( wp_unslash(
$_POST['mcs_item_nonce'] ) ), 'mcs_item_save' ) ) { return; }
  if ( ! current_user_can( 'edit_post', $post_id ) ) { return; }
  update_post_meta( $post_id, 'mcs_desc', sanitize_textarea_field( wp_unslash( isset(
$_POST['mcs_desc'] ) ? $_POST['mcs_desc'] : '' ) ) );
  update_post_meta( $post_id, 'mcs_price', sanitize_text_field( wp_unslash( isset( $_POST['mcs_price']
) ? $_POST['mcs_price'] : '' ) ) );
} );

/* Photo + price columns in the list, so the client sees everything at a glance. */
add_filter( 'manage_mcs_item_posts_columns', function ( $cols ) {
  return array( 'cb' => $cols['cb'], 'mcs_photo' => 'Photo', 'title' => 'Name', 'mcs_price' =>
'Price', 'taxonomy-mcs_cat' => 'Category', 'date' => 'Date' );
} );
add_action( 'manage_mcs_item_posts_custom_column', function ( $col, $id ) {
  if ( 'mcs_photo' === $col ) { echo get_the_post_thumbnail( $id, array( 50, 50 ) ); }
  if ( 'mcs_price' === $col ) { echo esc_html( get_post_meta( $id, 'mcs_price', true ) ); }
}, 10, 2 );

/* ================================================================== */
/* PART 5. SHORTCODE - shows the list on a page: [mcs_menu]           */
/* ================================================================== */

add_shortcode( 'mcs_menu', 'mcs_menu_html' );
function mcs_menu_html( $atts = array() ) {
  /* Options: [mcs_menu] shows everything, [mcs_menu category="coffee"] shows one category. */
  $atts      = shortcode_atts( array( 'category' => '' ), $atts );
  $card_tpl  = file_get_contents( MCS_DIR . 'templates/menu-card.html' );
  $group_tpl = file_get_contents( MCS_DIR . 'templates/menu-group.html' );
```

```
  $out = '';
  $args = array( 'taxonomy' => 'mcs_cat', 'hide_empty' => true, 'orderby' => 'name' );
  if ( $atts['category'] ) { $args['slug'] = sanitize_title( $atts['category'] ); }
  $cats = get_terms( $args );
  foreach ( $cats as $cat ) {
    $items = get_posts( array(
      'post_type'      => 'mcs_item',
      'posts_per_page' => -1,
      'orderby'        => array( 'menu_order' => 'ASC', 'title' => 'ASC' ),
      'tax_query'      => array( array( 'taxonomy' => 'mcs_cat', 'terms' => $cat->term_id ) ), //
phpcs:ignore
    ) );
    $cards = '';
    foreach ( $items as $item ) {
      $cards .= str_replace(
        array( '%IMG%', '%NAME%', '%DESC%', '%PRICE%' ),
        array(
          esc_url( get_the_post_thumbnail_url( $item->ID, 'medium_large' ) ),
          esc_html( get_the_title( $item ) ),
          esc_html( get_post_meta( $item->ID, 'mcs_desc', true ) ),
          esc_html( get_post_meta( $item->ID, 'mcs_price', true ) ),
        ),
        $card_tpl
      );
    }
    $out .= str_replace( array( '%CAT%', '%SLUG%', '%CARDS%' ), array( esc_html( $cat->name ),
esc_attr( $cat->slug ), $cards ), $group_tpl );
  }
  return '<div class="mcs-menu">' . $out . '</div>';
}

/* ================================================================== */
/* PART 6. SETTINGS PAGE - My Cafe > Settings                         */
/* ================================================================== */

add_action( 'admin_menu', function () {
  add_menu_page( 'My Cafe', 'My Cafe', 'manage_options', 'mcs-settings', 'mcs_settings_page',
'dashicons-store', 58 );
  add_submenu_page( 'mcs-settings', 'Settings', 'Settings', 'manage_options', 'mcs-settings',
'mcs_settings_page' );
  add_submenu_page( 'mcs-settings', 'Pictures', 'Pictures', 'manage_options', 'mcs-pictures',
'mcs_pictures_page' );
} );

function mcs_settings_page() {
  if ( ! current_user_can( 'manage_options' ) ) { return; }
  if ( isset( $_POST['mcs_save'] ) && check_admin_referer( 'mcs_settings' ) ) {
    update_option( 'mcs_settings', array(
      'phone'     => sanitize_text_field( wp_unslash( $_POST['phone'] ) ),
      'order_url' => esc_url_raw( wp_unslash( $_POST['order_url'] ) ),
    ) );
    echo '<div class="notice notice-success"><p>Saved.</p></div>';
  }
  if ( isset( $_POST['mcs_rebuild'] ) && check_admin_referer( 'mcs_settings' ) ) {
    mcs_setup( true );
    echo '<div class="notice notice-success"><p>Pages rebuilt from the original design.</p></div>';
  }
  echo '<div class="wrap"><h1>My Cafe - Settings</h1><form method="post">';
  wp_nonce_field( 'mcs_settings' );
  echo '<table class="form-table">';
  echo '<tr><th>Phone number</th><td><input class="regular-text" name="phone" value="' . esc_attr(
mcs_opt( 'phone' ) ) . '"><p class="description">Shown in the header and footer.</p></td></tr>';
  echo '<tr><th>Order online link</th><td><input class="regular-text" type="url" name="order_url"
value="' . esc_attr( mcs_opt( 'order_url' ) ) . '"><p class="description">Used by every "Order now"
```

```
button.</p></td></tr>';
  echo '</table><p><button class="button button-primary" name="mcs_save" value="1">Save</button></p>';
  echo '<h2>Tools</h2><p><button class="button" name="mcs_rebuild" value="1" onclick="return
confirm(\'Rebuild all pages? Your Elementor edits on them will be lost.\')">Rebuild pages from the
original design</button></p>';
  echo '</form></div>';
}

/* ================================================================== */
/* PART 7. PICTURES - import into Media + "Replace" page              */
/* ================================================================== */

function mcs_import_images() {
  require_once ABSPATH . 'wp-admin/includes/image.php';
  $map = get_option( 'mcs_images', array() );
  foreach ( glob( MCS_DIR . 'assets/img/*.*' ) as $path ) {
    $name = basename( $path );
    if ( isset( $map[ $name ]['id'] ) && get_post( $map[ $name ]['id'] ) ) { continue; }   // already
imported
    $up = wp_upload_bits( $name, null, file_get_contents( $path ) );
    if ( ! empty( $up['error'] ) ) { continue; }
    $type = wp_check_filetype( $name );
    $id   = wp_insert_attachment( array( 'post_mime_type' => $type['type'], 'post_title' => $name,
'post_status' => 'inherit' ), $up['file'] );
    wp_update_attachment_metadata( $id, wp_generate_attachment_metadata( $id, $up['file'] ) );
    $map[ $name ] = array( 'id' => $id, 'url' => $up['url'] );
  }
  update_option( 'mcs_images', $map, false );
}

function mcs_pictures_page() {
  if ( ! current_user_can( 'manage_options' ) ) { return; }
  $map = get_option( 'mcs_images', array() );
  if ( isset( $_POST['pic_name'], $_POST['pic_id'] ) && check_admin_referer( 'mcs_pics' ) ) {
    $name = sanitize_file_name( wp_unslash( $_POST['pic_name'] ) );
    $new  = wp_get_attachment_url( absint( $_POST['pic_id'] ) );
    if ( $new && isset( $map[ $name ] ) ) {
      $old = $map[ $name ]['url'];
      $map[ $name ] = array( 'id' => absint( $_POST['pic_id'] ), 'url' => $new );
      update_option( 'mcs_images', $map, false );
      mcs_replace_everywhere( $old, $new );
      echo '<div class="notice notice-success"><p>Picture replaced on every page.</p></div>';
    }
  }
  wp_enqueue_media();
  echo '<div class="wrap"><h1>My Cafe - Pictures</h1><p>Click Replace, choose a picture, done. It
changes on every page.</p>';
  echo '<form method="post" id="mcs-pic-form">';
  wp_nonce_field( 'mcs_pics' );
  echo '<input type="hidden" name="pic_name" id="pic_name"><input type="hidden" name="pic_id"
id="pic_id"></form>';
  echo '<div style="display:flex;flex-wrap:wrap;gap:16px">';
  foreach ( $map as $name => $m ) {
    echo '<div style="width:200px;background:#fff;padding:10px;border:1px solid #ddd">';
    echo '<img src="' . esc_url( $m['url'] ) . '" style="width:100%;height:120px;object-
fit:contain">';
    echo '<p style="font-size:12px">' . esc_html( $name ) . '</p>';
    echo '<button type="button" class="button mcs-replace" data-name="' . esc_attr( $name ) .
'">Replace</button></div>';
  }
  echo '</div></div>';
  echo "<script>jQuery(function($){ $('.mcs-replace').on('click',function(){ var
n=$(this).data('name'); var f=wp.media({title:'Replace '+n,multiple:false,library:{type:'image'}});
f.on('select',function(){ $('#pic_name').val(n);
```

```
$('#pic_id').val(f.state().get('selection').first().id); $('#mcs-pic-form').submit(); }); f.open();
}); });</script>";
}

/* Swap the old picture address for the new one inside every page we made. */
function mcs_replace_everywhere( $old, $new ) {
  $pages = get_posts( array( 'post_type' => 'page', 'posts_per_page' => -1, 'meta_key' => '_mcs_page',
'fields' => 'ids' ) ); // phpcs:ignore
  foreach ( $pages as $id ) {
    $data = json_decode( get_post_meta( $id, '_elementor_data', true ), true );
    if ( is_array( $data ) ) {
      array_walk_recursive( $data, function ( &$v ) use ( $old, $new ) { if ( is_string( $v ) ) { $v =
str_replace( $old, $new, $v ); } } );
      update_post_meta( $id, '_elementor_data', wp_slash( wp_json_encode( $data ) ) );
    }
    delete_post_meta( $id, '_elementor_css' );   // Elementor rebuilds its CSS
  }
}

/* ================================================================== */
/* PART 8. SETUP - runs when the plugin is activated                  */
/* ================================================================== */

register_activation_hook( __FILE__, 'mcs_setup' );

function mcs_setup( $rebuild = false ) {
  if ( ! get_option( 'permalink_structure' ) ) {   // nice addresses like /menu/ instead of ?
page_id=14
    update_option( 'permalink_structure', '/%postname%/' );
  }
  mcs_register_lists();          // make sure the list exists before adding items
  mcs_import_images();           // 1. pictures into Media
  mcs_seed_items();              // 2. starting menu items (only the first time)
  mcs_create_pages( $rebuild );  // 3. pages in Elementor
  add_option( 'mcs_settings', array( 'phone' => '+91 00000 00000', 'order_url' => '#' ) );
  flush_rewrite_rules();
}

/* Starting content from data/menu-seed.json, so the site never looks empty. */
function mcs_seed_items() {
  if ( get_option( 'mcs_seeded' ) ) { return; }   // never twice: client changes stay safe
  $map  = get_option( 'mcs_images', array() );
  $seed = json_decode( file_get_contents( MCS_DIR . 'data/menu-seed.json' ), true );
  foreach ( $seed as $item ) {
    $term = term_exists( $item['category'], 'mcs_cat' );
    if ( ! $term ) { $term = wp_insert_term( $item['category'], 'mcs_cat' ); }
    $id = wp_insert_post( array( 'post_type' => 'mcs_item', 'post_status' => 'publish', 'post_title'
=> $item['name'] ) );
    wp_set_object_terms( $id, array( (int) $term['term_id'] ), 'mcs_cat' );
    update_post_meta( $id, 'mcs_desc', $item['desc'] );
    update_post_meta( $id, 'mcs_price', $item['price'] );
    if ( isset( $map[ $item['photo'] ]['id'] ) ) { set_post_thumbnail( $id, $map[ $item['photo'] ]
['id'] ); }
  }
  update_option( 'mcs_seeded', 1 );
}

/* One Elementor section per HTML file (or a Shortcode widget for "[...]"). */
function mcs_elementor_sections( $slug, $files ) {
  $out = array();
  foreach ( $files as $n => $file ) {
    $id = substr( md5( $slug . $n ), 0, 7 );
    if ( '[' === $file[0] ) {
      $widget = array( 'id' => $id . 'w', 'elType' => 'widget', 'widgetType' => 'shortcode',
```

```
'settings' => array( 'shortcode' => $file ), 'elements' => array() );
    } else {
      $html   = mcs_fill( file_get_contents( MCS_DIR . 'pages/' . $slug . '/' . $file ) );
      $widget = array( 'id' => $id . 'w', 'elType' => 'widget', 'widgetType' => 'html', 'settings' =>
array( 'html' => $html ), 'elements' => array() );
    }
    $out[] = array(
      'id' => $id, 'elType' => 'section', 'isInner' => false,
      'settings' => array( 'layout' => 'full_width', 'gap' => 'no', 'css_classes' => 'mcs-sec',
'_title' => $file ),
      'elements' => array( array(
        'id' => $id . 'c', 'elType' => 'column', 'isInner' => false,
        'settings' => array( '_column_size' => 100 ),
        'elements' => array( $widget ),
      ) ),
    );
  }
  return $out;
}

function mcs_create_pages( $rebuild ) {
  $pages = json_decode( file_get_contents( MCS_DIR . 'data/pages.json' ), true );
  foreach ( $pages as $slug => $p ) {
    $page = get_page_by_path( $slug );
    if ( $page && ! $rebuild ) { continue; }                                    // page exists: keep
client edits
    if ( $page && ! get_post_meta( $page->ID, '_mcs_page', true ) ) { continue; } // not ours: never
touch
    $id = $page ? $page->ID : wp_insert_post( array( 'post_type' => 'page', 'post_status' =>
'publish', 'post_title' => $p['title_short'], 'post_name' => $slug ) );
    update_post_meta( $id, '_wp_page_template', 'elementor_canvas' );   // blank page, no theme header
    update_post_meta( $id, '_elementor_edit_mode', 'builder' );
    update_post_meta( $id, '_elementor_data', wp_slash( wp_json_encode( mcs_elementor_sections( $slug,
$p['sections'] ) ) ) );
    update_post_meta( $id, '_mcs_page', $slug );
    update_post_meta( $id, '_mcs_title', $p['seo_title'] );
    update_post_meta( $id, '_mcs_desc', $p['seo_desc'] );
    if ( 'home' === $slug ) { update_option( 'show_on_front', 'page' ); update_option(
'page_on_front', $id ); }
  }
}
```

APPENDIX B

Raining Berries: dynamic FAQ code

Tested on the Raining Berries plugin: same design and same click behaviour as the static FAQ, desktop and mobile.

templates/faq-wrap.html (new file)

```
<section data-pad="1" style="position: relative; background: rgb(255, 255, 255); padding: clamp(70px, 8vw, 120px) 28px;">
    <div style="max-width: 820px; margin: 0px auto;">
      <h2 data-reveal="1" style="margin: 0px 0px clamp(28px, 3vw, 44px); font-weight: 900; text-transform: uppercase; letter-spacing: -0.02em; line-height: 1; font-size: clamp(30px, 3.6vw, 48px); color: rgb(206, 14, 104);"><span class="sc-interp">%TITLE%</span></h2>
      <div data-reveal="stagger" style="display: flex; flex-direction: column; border-top: 1px solid rgba(42, 22, 32, 0.1);">

%ITEMS%

      </div>
    </div>
  </section>
```

templates/faq-item.html (new file)

```
<div style="border-bottom: 1px solid rgba(42, 22, 32, 0.1);">
<button data-i="%I%" data-rb-click="toggleFaq" aria-expanded="%OPEN%" style="width: 100%; min-height: 64px; display: flex; justify-content: space-between; align-items: center; gap: 20px; text-align: left; padding: 18px 0px; border: 0px; background: transparent; color: rgb(42, 22, 32); font-weight: 700; font-size: clamp(17px, 1.3vw, 19px); line-height: 1.35;">
<span><span class="sc-interp">%Q%</span></span>
<span style="flex: 0 0 auto; width: 32px; height: 32px; border-radius: 50%; border: 2px solid rgb(242, 23, 129); color: rgb(206, 14, 104); display: flex; align-items: center; justify-content: center; font-size: 20px; line-height: 1; font-weight: 500;"><span class="sc-interp">%SIGN%</span></span>
</button>
<div data-rbif="f.open" style="display: %SHOW%;">
<p style="margin: 0px; padding: 0px 48px 22px 0px; max-width: 66ch;"><span class="sc-interp">%A%
</span></p>
</div>
</div>
```

paste at the END of raining-berries-site.php

```
/* ================================================================== */
/* 6. FAQ (Dashboard > FAQs) - makes the home page FAQ dynamic        */
/* ================================================================== */

/* 1. The FAQs list in the dashboard (title = the question) */
add_action( 'init', function () {
  register_post_type( 'rb_faq', array(
    'labels'        => array( 'name' => 'FAQs', 'singular_name' => 'FAQ', 'add_new_item' => 'Add Question', 'edit_item' => 'Edit Question', 'all_items' => 'All Questions' ),
    'public'        => false,
    'show_ui'       => true,
    'menu_icon'     => 'dashicons-editor-help',
    'menu_position' => 22,
    'supports'      => array( 'title', 'page-attributes' ),
  ) );
```

```
} );

/* 2. The answer box */
add_action( 'add_meta_boxes', function () {
  add_meta_box( 'rb_faq_box', 'Answer', function ( $post ) {
    wp_nonce_field( 'rb_faq_save', 'rb_faq_nonce' );
    echo '<textarea name="rb_faq_answer" rows="5" style="width:100%">' . esc_textarea( get_post_meta(
$post->ID, 'rb_faq_answer', true ) ) . '</textarea>';
    echo '<p class="description">The title above is the question. Order: Page Attributes &gt; Order (1
= first).</p>';
  }, 'rb_faq', 'normal', 'high' );
} );

/* 3. Save the answer */
add_action( 'save_post_rb_faq', function ( $post_id ) {
  if ( ! isset( $_POST['rb_faq_nonce'] ) || ! wp_verify_nonce( sanitize_text_field( wp_unslash(
$_POST['rb_faq_nonce'] ) ), 'rb_faq_save' ) ) { return; }
  if ( ! current_user_can( 'edit_post', $post_id ) ) { return; }
  update_post_meta( $post_id, 'rb_faq_answer', sanitize_textarea_field( wp_unslash( isset(
$_POST['rb_faq_answer'] ) ? $_POST['rb_faq_answer'] : '' ) ) );
} );

/* 4. Copy the 5 questions from the design into the list (only once) */
add_action( 'admin_init', function () {
  if ( get_option( 'rb_faq_seeded' ) ) { return; }
  $start = array(
    array( 'What is Raining Berries?', 'Raining Berries is a health-focused café serving delicious
açaí bowls, smoothies, specialty coffee, gelato, sandwiches, and light bites. We’re passionate about
making better-for-you food and drinks delicious, approachable, and fun.' ),
    array( 'Where was Raining Berries founded?', 'Raining Berries was founded in Wesley Chapel,
Florida. We’ve continued to grow across the state and are excited to bring Raining Berries to new
communities and new states across the country.' ),
    array( 'What makes Raining Berries different?', 'We believe healthy food shouldn’t mean
sacrificing flavor. Our menu combines quality ingredients, bold flavors, and creative combinations to
give you options that fit a variety of lifestyles and cravings.' ),
    array( 'Where does Raining Berries coffee come from?', 'Our coffee is single-origin and ethically
sourced from Tanzania in East Africa.' ),
    array( 'Can I order online?', 'Yes! Online ordering is available! Visit our website or Raining
Berries Rewards app to place your order.' ),
  );
  foreach ( $start as $i => $qa ) {
    $id = wp_insert_post( array( 'post_type' => 'rb_faq', 'post_status' => 'publish', 'post_title' =>
$qa[0], 'menu_order' => $i + 1 ) );
    update_post_meta( $id, 'rb_faq_answer', $qa[1] );
  }
  update_option( 'rb_faq_seeded', 1 );
} );

/* 5. [rb_faq] prints the FAQ section with the exact design (first question open) */
add_shortcode( 'rb_faq', function () {
  $item  = file_get_contents( RB_DIR . 'templates/faq-item.html' );
  $items = '';
  $faqs  = get_posts( array( 'post_type' => 'rb_faq', 'posts_per_page' => -1, 'orderby' => array(
'menu_order' => 'ASC', 'title' => 'ASC' ) ) );
  foreach ( $faqs as $i => $f ) {
    $open   = ( 0 === $i );
    $items .= str_replace(
      array( '%I%', '%OPEN%', '%SIGN%', '%SHOW%', '%Q%', '%A%' ),
      array( $i, $open ? 'true' : 'false', $open ? '−' : '+', $open ? 'contents' : 'none', esc_html(
get_the_title( $f ) ), esc_html( get_post_meta( $f->ID, 'rb_faq_answer', true ) ) ),
      $item
    );
  }
  $wrap = file_get_contents( RB_DIR . 'templates/faq-wrap.html' );
```

```
    return str_replace( array( '%TITLE%', '%ITEMS%' ), array( 'Good to know', $items ), $wrap );
} );
```

APPENDIX C

Glossary

Word	Simple meaning
Plugin	A folder of files that adds features to WordPress.
Activation hook	Code that runs once when the plugin is activated.
Hook / action / filter	A fixed moment in WordPress where your code can run (action) or change a value (filter).
Enqueue	The WordPress way to add a CSS or JS file to a page.
Custom Post Type (CPT)	A new list in the dashboard, like Posts but for your content (Menu Items, Reviews).
Taxonomy	Categories (or tags) for a list.
Post meta	An extra value stored with one item (price, description).
Meta box	A box with fields on the edit screen of an item.
Option	A single saved value for the whole site (settings, picture list).
Shortcode	A word in brackets like [mcs_menu] that is replaced by PHP output.
Template	An HTML file with blanks (%NAME%) that PHP fills once per item.
Placeholder	A word in double brackets like {{HOME}} replaced by a real value.
Slug	The short web name of a page or category: "menu", "coffee".
Nonce	A security token that proves a form came from your own screen.
Sanitize / escape	Clean data before saving it / make it safe before showing it.
Elementor Canvas	A page template with no theme header or footer.
Staging site	A private copy of the site for testing.
JSON	A simple text format for lists and settings: {"key": "value"}.

APPENDIX D

One-page checklist for every new client

1. Plan: pages, sections, static vs dynamic, fields, settings (Chapter 4).
2. Create the folder structure with the client prefix (Chapter 5).
3. Main file header, constants, helpers (Chapters 6-7).
4. CSS/JS loading + theme styles off; header and footer (Chapters 8-9).
5. Cut the design into section files; write pages.json (Chapters 10-11).
6. Pictures import + Pictures screen (Chapter 12).
7. One CPT per dynamic list, fields, columns (Chapter 13).
8. Templates + shortcode per list (Chapter 14).
9. Settings screen; starting content (Chapters 15-16).
10. JavaScript, SEO, mobile (Chapter 17).
11. Zip, install on staging, test everything (Chapters 18, 23).
12. Client guide + handover (Chapter 25).